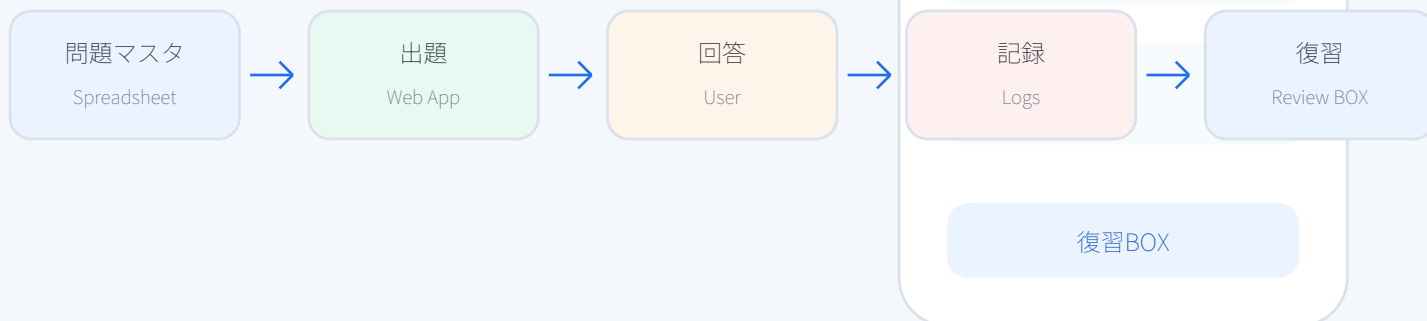


Firestore理解度チェック

240問 解答・解説集

問題マスタの全問に、正解・ヒント・解説を付けた学習用PDF

Firestore学習の流れ



収録内容

カテゴリ別の重要ポイント / 4択問題 / 正解番号 / 正解テキスト / ヒント / 解説 / タグ
スマホ学習サイトと併用しやすいよう、復習しやすいカード形式で整理

カテゴリ別の問題数

240問をカテゴリ別に確認できます。棒の長さは問題数です。



使い方

- 1 **まず解く**
問題文とコード例から、Firestore APIの役割を判断する。
- 2 **答えを見る**
正解番号だけでなく、正解テキストを声に出して確認する。
- 3 **解説を読む**
なぜそのAPIや設計が合うのかを、実装場面と結び付ける。
- 4 **復習する**
間違えたカテゴリだけを復習BOXとして重点的に回す。

基本操作

10問 / 正解番号・正解テキスト・ヒント・解説付き

001 FSQ-001

基本操作

初級

日替わり

Firestore API 「addDoc」 の説明として正しいものは？

OK 1. コレクションへ自動IDで新規ドキュメントを追加する

2. CSSの見た目を変更する

3. HTML要素を自動生成する

4. Firebaseプロジェクトを削除する

正解: 1. コレクションへ自動IDで新規ドキュメントを追加する

ヒント

API名の動詞と、取得対象が1件か複数件かに注目します。

解説

addDocはFirestore操作の基本APIです。新規追加、取得、更新、削除、参照作成など、役割を分けて覚えると実装判断がしやすくなります。

002 FSQ-002

基本操作

初級

weak

Firestore API 「getDoc」 の説明として正しいものは？

1. Firebaseプロジェクトを削除する

OK 2. 特定ドキュメントを1件だけ取得する

3. CSSの見た目を変更する

4. HTML要素を自動生成する

正解: 2. 特定ドキュメントを1件だけ取得する

ヒント

API名の動詞と、取得対象が1件か複数件かに注目します。

解説

getDocはFirestore操作の基本APIです。新規追加、取得、更新、削除、参照作成など、役割を分けて覚えると実装判断がしやすくなります。

003 FSQ-003

基本操作

中級

日替わり

Firestore API 「getDocs」 の説明として正しいものは？

1. CSSの見た目を変更する

2. HTML要素を自動生成する

OK 3. クエリやコレクションの結果を一覧取得する

4. Firebaseプロジェクトを削除する

正解: 3. クエリやコレクションの結果を一覧取得する

ヒント

API名の動詞と、取得対象が1件か複数件かに注目します。

解説

getDocsはFirestore操作の基本APIです。新規追加、取得、更新、削除、参照作成など、役割を分けて覚えると実装判断がしやすくなります。

004 FSQ-004

基本操作

中級

日替わり

Firestore API 「setDoc」 の説明として正しいものは？

1. CSSの見た目を変更する

2. HTML要素を自動生成する

3. Firebaseプロジェクトを削除する

OK 4. 指定IDのドキュメントを作成または上書きする

正解: 4. 指定IDのドキュメントを作成または上書きする

ヒント

API名の動詞と、取得対象が1件か複数件かに注目します。

解説

setDocはFirestore操作の基本APIです。新規追加、取得、更新、削除、参照作成など、役割を分けて覚えると実装判断がしやすくなります。

005 FSQ-005

基本操作

応用

日替わり

Firestore API 「updateDoc」 の説明として正しいものは？

OK 1. 既存ドキュメントの一部フィールドを更新する

2. CSSの見た目を変更する

3. HTML要素を自動生成する

4. Firebaseプロジェクトを削除する

正解: 1. 既存ドキュメントの一部フィールドを更新する

ヒント

API名の動詞と、取得対象が1件か複数件かに注目します。

解説

updateDocはFirestore操作の基本APIです。新規追加、取得、更新、削除、参照作成など、役割を分けて覚えると実装判断がしやすくなります。

006 FSQ-006

基本操作

初級

日替わり

Firestore API 「deleteDoc」 の説明として正しいものは？

1. CSSの見た目を変更する

OK 2. 指定したドキュメントを削除する

3. HTML要素を自動生成する

4. Firebaseプロジェクトを削除する

正解: 2. 指定したドキュメントを削除する

ヒント

API名の動詞と、取得対象が1件か複数件かに注目します。

解説

deleteDocはFirestore操作の基本APIです。新規追加、取得、更新、削除、参照作成など、役割を分けて覚えると実装判断がしやすくなります。

007 FSQ-007

基本操作

初級

日替わり

Firestore API 「collection」の説明として正しいものは？

1. CSSの見た目を変更する

2. HTML要素を自動生成する

OK 3. コレクション参照を作る

4. Firebaseプロジェクトを削除する

正解: 3. コレクション参照を作る

ヒント

API名の動詞と、取得対象が1件か複数件かに注目します。

解説

collectionはFirestore操作の基本APIです。新規追加、取得、更新、削除、参照作成など、役割を分けて覚えると実装判断がしやすくなります。

008 FSQ-008

基本操作

中級

日替わり

Firestore API 「doc」の説明として正しいものは？

1. CSSの見た目を変更する

2. HTML要素を自動生成する

3. Firebaseプロジェクトを削除する

OK 4. ドキュメント参照を作る

正解: 4. ドキュメント参照を作る

ヒント

API名の動詞と、取得対象が1件か複数件かに注目します。

解説

docはFirestore操作の基本APIです。新規追加、取得、更新、削除、参照作成など、役割を分けて覚えると実装判断がしやすくなります。

009 FSQ-009

基本操作

中級

日替わり

Firestore API 「query」 の説明として正しいものは？

OK 1. コレクション参照に検索条件を組み合わせる

2. CSSの見た目を変更する

3. HTML要素を自動生成する

4. Firebaseプロジェクトを削除する

正解: 1. コレクション参照に検索条件を組み合わせる

ヒント

API名の動詞と、取得対象が1件か複数件かに注目します。

解説

queryはFirestore操作の基本APIです。新規追加、取得、更新、削除、参照作成など、役割を分けて覚えると実装判断がしやすくなります。

010 FSQ-010

基本操作

応用

daily

Firestore API 「where」 の説明として正しいものは？

1. CSSの見た目を変更する

2. HTML要素を自動生成する

3. Firebaseプロジェクトを削除する

OK 4. フィールド条件で絞り込む

正解: 4. フィールド条件で絞り込む

ヒント

API名の動詞と、取得対象が1件か複数件かに注目します。

解説

whereはFirestore操作の基本APIです。新規追加、取得、更新、削除、参照作成など、役割を分けて覚えると実装判断がしやすくなります。

参照

8問 / 正解番号・正解テキスト・ヒント・解説付き

011 FSQ-011

参照

初級

全範囲

ordersコレクション内の特定IDのドキュメントを更新したい。正しい参照の作り方は？

```
const ref = doc(db, 'orders', docId);
```

1. collection(db, 'orders', docId)

2. query(db, 'orders', docId)

OK 3. doc(db, 'orders', docId)

4. where('orders', '==', docId)

正解: 3. doc(db, 'orders', docId)

ヒント

1件のドキュメントを直接指すときはdocを使います。

解説

updateDocやdeleteDocにはDocumentReferenceを渡します。collectionはコレクション全体を指す参照です。

012 FSQ-012

参照

初級

全範囲

menusコレクション内の特定IDのドキュメントを更新したい。正しい参照の作り方は？

```
const ref = doc(db, 'menus', docId);
```

1. collection(db, 'menus', docId)

2. query(db, 'menus', docId)

3. where('menus', '==', docId)

OK 4. doc(db, 'menus', docId)

正解: 4. doc(db, 'menus', docId)

ヒント

1件のドキュメントを直接指すときはdocを使います。

解説

updateDocやdeleteDocにはDocumentReferenceを渡します。collectionはコレクション全体を指す参照です。

013 FSQ-013

参照

中級

全範囲

usersコレクション内の特定IDのドキュメントを更新したい。正しい参照の作り方は？

```
const ref = doc(db, 'users', docId);
```

OK 1. doc(db, 'users', docId)

2. collection(db, 'users', docId)

3. query(db, 'users', docId)

4. where('users', '==', docId)

正解: 1. doc(db, 'users', docId)

ヒント

1件のドキュメントを直接指すときはdocを使います。

解説

updateDocやdeleteDocにはDocumentReferenceを渡します。collectionはコレクション全体を指す参照です。

014 FSQ-014

参照

中級

全範囲

CookListコレクション内の特定IDのドキュメントを更新したい。正しい参照の作り方は？

```
const ref = doc(db, 'CookList', docId);
```

1. collection(db, 'CookList', docId)

OK 2. doc(db, 'CookList', docId)

3. query(db, 'CookList', docId)

4. where('CookList', '==', docId)

正解: 2. doc(db, 'CookList', docId)

ヒント

1件のドキュメントを直接指すときはdocを使います。

解説

updateDocやdeleteDocにはDocumentReferenceを渡します。collectionはコレクション全体を指す参照です。

015 FSQ-015

参照

応用

全範囲

StaffListコレクション内の特定IDのドキュメントを更新したい。正しい参照の作り方は？

```
const ref = doc(db, 'StaffList', docId);
```

1. collection(db, 'StaffList', docId)

2. query(db, 'StaffList', docId)

OK 3. doc(db, 'StaffList', docId)

4. where('StaffList', '==', docId)

正解: 3. doc(db, 'StaffList', docId)

ヒント

1件のドキュメントを直接指すときはdocを使います。

解説

updateDocやdeleteDocにはDocumentReferenceを渡します。collectionはコレクション全体を指す参照です。

016 FSQ-016

参照

初級

全範囲

quizQuestionsコレクション内の特定IDのドキュメントを更新したい。正しい参照の作り方は？

```
const ref = doc(db, 'quizQuestions', docId);
```

1. collection(db, 'quizQuestions', docId)

2. query(db, 'quizQuestions', docId)

3. where('quizQuestions', '==', docId)

OK 4. doc(db, 'quizQuestions', docId)

正解: 4. doc(db, 'quizQuestions', docId)

ヒント

1件のドキュメントを直接指すときはdocを使います。

解説

updateDocやdeleteDocにはDocumentReferenceを渡します。collectionはコレクション全体を指す参照です。

017 FSQ-017

参照

初級

全範囲

learningLogsコレクション内の特定IDのドキュメントを更新したい。正しい参照の作り方は？

```
const ref = doc(db, 'learningLogs', docId);
```

OK 1. doc(db, 'learningLogs', docId)

2. collection(db, 'learningLogs', docId)

3. query(db, 'learningLogs', docId)

4. where('learningLogs', '==', docId)

正解: 1. doc(db, 'learningLogs', docId)

ヒント

1件のドキュメントを直接指すときはdocを使います。

解説

updateDocやdeleteDocにはDocumentReferenceを渡します。collectionはコレクション全体を指す参照です。

018 FSQ-018

参照

中級

全範囲

settingsコレクション内の特定IDのドキュメントを更新したい。正しい参照の作り方は？

```
const ref = doc(db, 'settings', docId);
```

1. collection(db, 'settings', docId)

OK 2. doc(db, 'settings', docId)

3. query(db, 'settings', docId)

4. where('settings', '==', docId)

正解: 2. doc(db, 'settings', docId)

ヒント

1件のドキュメントを直接指すときはdocを使います。

解説

updateDocやdeleteDocにはDocumentReferenceを渡します。collectionはコレクション全体を指す参照です。

追加

8問 / 正解番号・正解テキスト・ヒント・解説付き

019 FSQ-019

追加

中級

全範囲

ordersコレクションへ自動IDで新しいドキュメントを追加したい。使うAPIは？

```
await addDoc(collection(db, 'orders'), data);
```

1. getDocs

2. deleteDoc

OK 3. addDoc

4. where

正解: 3. addDoc

ヒント

自動IDで追加する操作です。

解説

addDocは指定したコレクションに自動IDのドキュメントを追加します。注文登録や学習ログ保存のような新規データ作成に向いています。

020 FSQ-020

追加

応用

全範囲

menusコレクションへ自動IDで新しいドキュメントを追加したい。使うAPIは？

```
await addDoc(collection(db, 'menus'), data);
```

1. getDocs

2. deleteDoc

3. where

OK 4. addDoc

正解: 4. addDoc

ヒント

自動IDで追加する操作です。

解説

addDocは指定したコレクションに自動IDのドキュメントを追加します。注文登録や学習ログ保存のような新規データ作成に向いています。

021 FSQ-021

追加

初級

全範囲

usersコレクションへ自動IDで新しいドキュメントを追加したい。使うAPIは？

```
await addDoc(collection(db, 'users'), data);
```

OK 1. addDoc

2. getDocs

3. deleteDoc

4. where

正解: 1. addDoc

ヒント

自動IDで追加する操作です。

解説

addDocは指定したコレクションに自動IDのドキュメントを追加します。注文登録や学習ログ保存のような新規データ作成に向いています。

022 FSQ-022

追加

初級

全範囲

CookListコレクションへ自動IDで新しいドキュメントを追加したい。使うAPIは？

```
await addDoc(collection(db, 'CookList'), data);
```

1. getDocs

OK 2. addDoc

3. deleteDoc

4. where

正解: 2. addDoc

ヒント

自動IDで追加する操作です。

解説

addDocは指定したコレクションに自動IDのドキュメントを追加します。注文登録や学習ログ保存のような新規データ作成に向いています。

023 FSQ-023

追加

中級

全範囲

StaffListコレクションへ自動IDで新しいドキュメントを追加したい。使うAPIは？

```
await addDoc(collection(db, 'StaffList'), data);
```

1. getDocs

2. deleteDoc

OK 3. addDoc

4. where

正解: 3. addDoc

ヒント

自動IDで追加する操作です。

解説

addDocは指定したコレクションに自動IDのドキュメントを追加します。注文登録や学習ログ保存のような新規データ作成に向いています。

024 FSQ-024

追加

中級

全範囲

quizQuestionsコレクションへ自動IDで新しいドキュメントを追加したい。使うAPIは？

```
await addDoc(collection(db, 'quizQuestions'), data);
```

1. getDocs

2. deleteDoc

3. where

OK 4. addDoc

正解: 4. addDoc

ヒント

自動IDで追加する操作です。

解説

addDocは指定したコレクションに自動IDのドキュメントを追加します。注文登録や学習ログ保存のような新規データ作成に向いています。

025 FSQ-025

追加

応用

全範囲

learningLogsコレクションへ自動IDで新しいドキュメントを追加したい。使うAPIは？

```
await addDoc(collection(db, 'learningLogs'), data);
```

OK 1. addDoc

2. getDocs

3. deleteDoc

4. where

正解: 1. addDoc

ヒント

自動IDで追加する操作です。

解説

addDocは指定したコレクションに自動IDのドキュメントを追加します。注文登録や学習ログ保存のような新規データ作成に向いています。

026 FSQ-026

追加

初級

全範囲

settingsコレクションへ自動IDで新しいドキュメントを追加したい。使うAPIは？

```
await addDoc(collection(db, 'settings'), data);
```

1. getDocs

OK 2. addDoc

3. deleteDoc

4. where

正解: 2. addDoc

ヒント

自動IDで追加する操作です。

解説

addDocは指定したコレクションに自動IDのドキュメントを追加します。注文登録や学習ログ保存のような新規データ作成に向いています。

更新

18問 / 正解番号・正解テキスト・ヒント・解説付き

027 FSQ-027

更新

初級

全範囲

既存ドキュメントの「status」フィールドだけを変更したい。最も適切なAPIは？

```
await updateDoc(ref, { status: value });
```

1. addDoc

2. getDocs

OK 3. updateDoc

4. collection

正解: 3. updateDoc

ヒント

一部フィールドだけを変更する操作です。

解説

updateDocは既存ドキュメントの指定フィールドだけを更新できます。ドキュメント全体を作り直す必要はありません。

028 FSQ-028

更新

中級

全範囲

既存ドキュメントの「table」フィールドだけを変更したい。最も適切なAPIは？

```
await updateDoc(ref, { table: value });
```

1. addDoc

2. getDocs

3. collection

OK 4. updateDoc

正解: 4. updateDoc

ヒント

一部フィールドだけを変更する操作です。

解説

updateDocは既存ドキュメントの指定フィールドだけを更新できます。ドキュメント全体を作り直す必要はありません。

029 FSQ-029

更新

中級

全範囲

既存ドキュメントの「guestCount」フィールドだけを変更したい。最も適切なAPIは？

```
await updateDoc(ref, { guestCount: value });
```

OK 1. updateDoc

2. addDoc

3. getDocs

4. collection

正解: 1. updateDoc

ヒント

一部フィールドだけを変更する操作です。

解説

updateDocは既存ドキュメントの指定フィールドだけを更新できます。ドキュメント全体を作り直す必要はありません。

030 FSQ-030

更新

応用

全範囲

既存ドキュメントの「staffCode」フィールドだけを変更したい。最も適切なAPIは？

```
await updateDoc(ref, { staffCode: value });
```

1. addDoc

OK 2. updateDoc

3. getDocs

4. collection

正解: 2. updateDoc

ヒント

一部フィールドだけを変更する操作です。

解説

updateDocは既存ドキュメントの指定フィールドだけを更新できます。ドキュメント全体を作り直す必要はありません。

031 FSQ-031

更新

初級

全範囲

既存ドキュメントの「createdAt」フィールドだけを変更したい。最も適切なAPIは？

```
await updateDoc(ref, { createdAt: value });
```

1. addDoc

2. getDocs

OK 3. updateDoc

4. collection

正解: 3. updateDoc

ヒント

一部フィールドだけを変更する操作です。

解説

updateDocは既存ドキュメントの指定フィールドだけを更新できます。ドキュメント全体を作り直す必要はありません。

032 FSQ-032

更新

初級

weak

既存ドキュメントの「updatedAt」フィールドだけを変更したい。最も適切なAPIは？

```
await updateDoc(ref, { updatedAt: value });
```

OK 1. updateDoc

2. getDocs

3. addDoc

4. collection

正解: 1. updateDoc

ヒント

一部フィールドだけを変更する操作です。

解説

updateDocは既存ドキュメントの指定フィールドだけを更新できます。ドキュメント全体を作り直す必要はありません。

033 FSQ-033

更新

中級

全範囲

既存ドキュメントの「priority」フィールドだけを変更したい。最も適切なAPIは？

```
await updateDoc(ref, { priority: value });
```

OK 1. updateDoc

2. addDoc

3. getDocs

4. collection

正解: 1. updateDoc

ヒント

一部フィールドだけを変更する操作です。

解説

updateDocは既存ドキュメントの指定フィールドだけを更新できます。ドキュメント全体を作り直す必要はありません。

034 FSQ-034

更新

中級

全範囲

既存ドキュメントの「orderMethod」フィールドだけを変更したい。最も適切なAPIは？

```
await updateDoc(ref, { orderMethod: value });
```

1. addDoc

OK 2. updateDoc

3. getDocs

4. collection

正解: 2. updateDoc

ヒント

一部フィールドだけを変更する操作です。

解説

updateDocは既存ドキュメントの指定フィールドだけを更新できます。ドキュメント全体を作り直す必要はありません。

035 FSQ-035

更新

応用

全範囲

既存ドキュメントの「answeredAt」フィールドだけを変更したい。最も適切なAPIは？

```
await updateDoc(ref, { answeredAt: value });
```

1. addDoc

2. getDocs

OK 3. updateDoc

4. collection

正解: 3. updateDoc

ヒント

一部フィールドだけを変更する操作です。

解説

updateDocは既存ドキュメントの指定フィールドだけを更新できます。ドキュメント全体を作り直す必要はありません。

036 FSQ-036

更新

初級

全範囲

既存ドキュメントの「category」フィールドだけを変更したい。最も適切なAPIは？

```
await updateDoc(ref, { category: value });
```

1. addDoc

2. getDocs

3. collection

OK 4. updateDoc

正解: 4. updateDoc

ヒント

一部フィールドだけを変更する操作です。

解説

updateDocは既存ドキュメントの指定フィールドだけを更新できます。ドキュメント全体を作り直す必要はありません。

037 FSQ-227

更新

初級

全範囲

Firestoreの更新値「increment(1)」について正しい説明は？

```
{ field: increment(1) }
```

1. クエリの並び順を必ず降順にする

2. Security Rulesを無効化する

OK 3. 数値フィールドを1増やす

4. 全コレクションを削除する

正解: 3. 数値フィールドを1増やす

ヒント

更新時に使える特殊な値があります。

解説

Firestoreには数値加算、配列操作、サーバー時刻、フィールド削除など、更新時に使える特殊な値があります。

038 FSQ-228

更新

中級

全範囲

Firestoreの更新値「increment(-1)」について正しい説明は？

```
{ field: increment(-1) }
```

1. クエリの並び順を必ず降順にする

2. Security Rulesを無効化する

3. 全コレクションを削除する

OK 4. 数値フィールドを1減らす

正解: 4. 数値フィールドを1減らす

ヒント

更新時に使える特殊な値があります。

解説

Firestoreには数値加算、配列操作、サーバー時刻、フィールド削除など、更新時に使える特殊な値があります。

039 FSQ-229

更新

中級

全範囲

Firestoreの更新値「arrayUnion(value)」について正しい説明は？

```
{ field: arrayUnion(value) }
```

OK 1. 配列に値を追加する

2. クエリの並び順を必ず降順にする

3. Security Rulesを無効化する

4. 全コレクションを削除する

正解: 1. 配列に値を追加する

ヒント

更新時に使える特殊な値があります。

解説

Firestoreには数値加算、配列操作、サーバー時刻、フィールド削除など、更新時に使える特殊な値があります。

040 FSQ-230

更新

応用

全範囲

Firestoreの更新値「arrayRemove(value)」について正しい説明は？

```
{ field: arrayRemove(value) }
```

1. クエリの並び順を必ず降順にする

OK 2. 配列から値を削除する

3. Security Rulesを無効化する

4. 全コレクションを削除する

正解: 2. 配列から値を削除する

ヒント

更新時に使える特殊な値があります。

解説

Firestoreには数値加算、配列操作、サーバー時刻、フィールド削除など、更新時に使える特殊な値があります。

041 FSQ-231

更新

初級

全範囲

Firestoreの更新値「serverTimestamp()」について正しい説明は？

```
{ field: serverTimestamp() }
```

1. クエリの並び順を必ず降順にする

2. Security Rulesを無効化する

OK 3. サーバー時刻を保存する

4. 全コレクションを削除する

正解: 3. サーバー時刻を保存する

ヒント

更新時に使える特殊な値があります。

解説

Firestoreには数値加算、配列操作、サーバー時刻、フィールド削除など、更新時に使える特殊な値があります。

042 FSQ-232

更新

初級

全範囲

Firestoreの更新値「deleteField()」について正しい説明は？

```
{ field: deleteField() }
```

1. クエリの並び順を必ず降順にする

2. Security Rulesを無効化する

3. 全コレクションを削除する

OK 4. 指定フィールドを削除する

正解: 4. 指定フィールドを削除する

ヒント

更新時に使える特殊な値があります。

解説

Firestoreには数値加算、配列操作、サーバー時刻、フィールド削除など、更新時に使える特殊な値があります。

043 FSQ-233

更新

中級

全範囲

Firestoreの更新値「Timestamp.now()」について正しい説明は？

```
{ field: Timestamp.now() }
```

OK 1. Timestamp値を作る

2. クエリの並び順を必ず降順にする

3. Security Rulesを無効化する

4. 全コレクションを削除する

正解: 1. Timestamp値を作る

ヒント

更新時に使える特殊な値があります。

解説

Firestoreには数値加算、配列操作、サーバー時刻、フィールド削除など、更新時に使える特殊な値があります。

044 FSQ-234

更新

中級

全範囲

Firestoreの更新値「GeoPoint(lat,lng)」について正しい説明は？

```
{ field: GeoPoint(lat,lng) }
```

1. クエリの並び順を必ず降順にする

OK 2. 緯度経度をGeoPointとして保存する

3. Security Rulesを無効化する

4. 全コレクションを削除する

正解: 2. 緯度経度をGeoPointとして保存する

ヒント

更新時に使える特殊な値があります。

解説

Firestoreには数値加算、配列操作、サーバー時刻、フィールド削除など、更新時に使える特殊な値があります。

クエリ

36問 / 正解番号・正解テキスト・ヒント・解説付き

045 FSQ-037

クエリ

初級

日替わり

statusが特定の値のドキュメントだけを取得したい。使う条件は？

```
query(collection(db, 'orders'), where('status', '==', value));
```

OK 1. where('status', '==', value)

2. orderBy('status')だけ

3. limit('status')

4. doc(db, 'status')

正解: 1. where('status', '==', value)

ヒント

条件で絞り込むAPIを選びます。

解説

whereはフィールド条件でクエリ結果を絞り込みます。orderByは並び替え、limitは件数制限です。

046 FSQ-038

クエリ

中級

日替わり

tableが特定の値のドキュメントだけを取得したい。使う条件は？

```
query(collection(db, 'orders'), where('table', '==', value));
```

1. orderBy('table')だけ

OK 2. where('table', '==', value)

3. limit('table')

4. doc(db, 'table')

正解: 2. where('table', '==', value)

ヒント

条件で絞り込むAPIを選びます。

解説

whereはフィールド条件でクエリ結果を絞り込みます。orderByは並び替え、limitは件数制限です。

047 FSQ-039

クエリ

中級

日替わり

guestCountが特定の値のドキュメントだけを取得したい。使う条件は？

```
query(collection(db, 'orders'), where('guestCount', '==', value));
```

1. orderBy('guestCount')だけ

2. limit('guestCount')

OK 3. where('guestCount', '==', value)

4. doc(db, 'guestCount')

正解: 3. where('guestCount', '==', value)

ヒント

条件で絞り込むAPIを選びます。

解説

whereはフィールド条件でクエリ結果を絞り込みます。orderByは並び替え、limitは件数制限です。

048 FSQ-040

クエリ

応用

日替わり

staffCodeが特定の値のドキュメントだけを取得したい。使う条件は？

```
query(collection(db, 'orders'), where('staffCode', '==', value));
```

1. orderBy('staffCode')だけ

2. limit('staffCode')

3. doc(db, 'staffCode')

OK 4. where('staffCode', '==', value)

正解: 4. where('staffCode', '==', value)

ヒント

条件で絞り込むAPIを選びます。

解説

whereはフィールド条件でクエリ結果を絞り込みます。orderByは並び替え、limitは件数制限です。

049 FSQ-041

クエリ

初級

日替わり

createdAtが特定の値のドキュメントだけを取得したい。使う条件は？

```
query(collection(db, 'orders'), where('createdAt', '==', value));
```

OK 1. where('createdAt', '==', value)

2. orderBy('createdAt')だけ

3. limit('createdAt')

4. doc(db, 'createdAt')

正解: 1. where('createdAt', '==', value)

ヒント

条件で絞り込むAPIを選びます。

解説

whereはフィールド条件でクエリ結果を絞り込みます。orderByは並び替え、limitは件数制限です。

050 FSQ-042

クエリ

初級

日替わり

updatedAtが特定の値のドキュメントだけを取得したい。使う条件は？

```
query(collection(db, 'orders'), where('updatedAt', '==', value));
```

1. orderBy('updatedAt')だけ

OK 2. where('updatedAt', '==', value)

3. limit('updatedAt')

4. doc(db, 'updatedAt')

正解: 2. where('updatedAt', '==', value)

ヒント

条件で絞り込むAPIを選びます。

解説

whereはフィールド条件でクエリ結果を絞り込みます。orderByは並び替え、limitは件数制限です。

051 FSQ-043

クエリ

中級

日替わり

priorityが特定の値のドキュメントだけを取得したい。使う条件は？

```
query(collection(db, 'orders'), where('priority', '==', value));
```

1. orderBy('priority')だけ

2. limit('priority')

OK 3. where('priority', '==', value)

4. doc(db, 'priority')

正解: 3. where('priority', '==', value)

ヒント

条件で絞り込むAPIを選びます。

解説

whereはフィールド条件でクエリ結果を絞り込みます。orderByは並び替え、limitは件数制限です。

052 FSQ-044

クエリ

中級

日替わり

orderMethodが特定の値のドキュメントだけを取得したい。使う条件は？

```
query(collection(db, 'orders'), where('orderMethod', '==', value));
```

1. orderBy('orderMethod')だけ

2. limit('orderMethod')

3. doc(db, 'orderMethod')

OK 4. where('orderMethod', '==', value)

正解: 4. where('orderMethod', '==', value)

ヒント

条件で絞り込むAPIを選びます。

解説

whereはフィールド条件でクエリ結果を絞り込みます。orderByは並び替え、limitは件数制限です。

053 FSQ-045

クエリ

応用

日替わり

answeredAtが特定の値のドキュメントだけを取得したい。使う条件は？

```
query(collection(db, 'orders'), where('answeredAt', '==', value));
```

OK 1. where('answeredAt', '==', value)

2. orderBy('answeredAt')だけ

3. limit('answeredAt')

4. doc(db, 'answeredAt')

正解: 1. where('answeredAt', '==', value)

ヒント

条件で絞り込むAPIを選びます。

解説

whereはフィールド条件でクエリ結果を絞り込みます。orderByは並び替え、limitは件数制限です。

054 FSQ-046

クエリ

初級

日替わり

categoryが特定の値のドキュメントだけを取得したい。使う条件は？

```
query(collection(db, 'orders'), where('category', '==', value));
```

1. orderBy('category')だけ

OK 2. where('category', '==', value)

3. limit('category')

4. doc(db, 'category')

正解: 2. where('category', '==', value)

ヒント

条件で絞り込むAPIを選びます。

解説

whereはフィールド条件でクエリ結果を絞り込みます。orderByは並び替え、limitは件数制限です。

055 FSQ-161

クエリ

初級

日替わり

statusで昇順に並べて取得したい。Firestoreクエリで使うものは？

```
query(collection(db, 'orders'), orderBy('status', 'asc'));
```

OK 1. orderBy('status', 'asc')

2. where('status', 'asc')

3. limit('status')

4. arrayUnion('status')

正解: 1. orderBy('status', 'asc')

ヒント

並び替えはorderByです。

解説

orderByは指定フィールドで結果を並び替えます。whereと組み合わせる場合はインデックスが必要になることがあります。

056 FSQ-162

クエリ

初級

日替わり

tableで昇順に並べて取得したい。Firestoreクエリで使うものは？

```
query(collection(db, 'orders'), orderBy('table', 'asc'));
```

1. where('table', 'asc')

OK 2. orderBy('table', 'asc')

3. limit('table')

4. arrayUnion('table')

正解: 2. orderBy('table', 'asc')

ヒント

並び替えはorderByです。

解説

orderByは指定フィールドで結果を並び替えます。whereと組み合わせる場合はインデックスが必要になることがあります。

057 FSQ-163

クエリ

中級

日替わり

guestCountで昇順に並べて取得したい。Firestoreクエリで使うものは？

```
query(collection(db, 'orders'), orderBy('guestCount', 'asc'));
```

1. where('guestCount', 'asc')

2. limit('guestCount')

OK 3. orderBy('guestCount', 'asc')

4. arrayUnion('guestCount')

正解: 3. orderBy('guestCount', 'asc')

ヒント

並び替えはorderByです。

解説

orderByは指定フィールドで結果を並び替えます。whereと組み合わせる場合はインデックスが必要になることがあります。

058 FSQ-164

クエリ

中級

日替わり

staffCodeで昇順に並べて取得したい。Firestoreクエリで使うものは？

```
query(collection(db, 'orders'), orderBy('staffCode', 'asc'));
```

1. where('staffCode', 'asc')

2. limit('staffCode')

3. arrayUnion('staffCode')

OK 4. orderBy('staffCode', 'asc')

正解: 4. orderBy('staffCode', 'asc')

ヒント

並び替えはorderByです。

解説

orderByは指定フィールドで結果を並び替えます。whereと組み合わせる場合はインデックスが必要になることがあります。

059 FSQ-165

クエリ

応用

日替わり

createdAtで昇順に並べて取得したい。Firestoreクエリで使うものは？

```
query(collection(db, 'orders'), orderBy('createdAt', 'asc'));
```

OK 1. orderBy('createdAt', 'asc')

2. where('createdAt', 'asc')

3. limit('createdAt')

4. arrayUnion('createdAt')

正解: 1. orderBy('createdAt', 'asc')

ヒント

並び替えはorderByです。

解説

orderByは指定フィールドで結果を並び替えます。whereと組み合わせる場合はインデックスが必要になることがあります。

060 FSQ-166

クエリ

初級

日替わり

updatedAtで昇順に並べて取得したい。Firestoreクエリで使うものは？

```
query(collection(db, 'orders'), orderBy('updatedAt', 'asc'));
```

1. where('updatedAt', 'asc')

OK 2. orderBy('updatedAt', 'asc')

3. limit('updatedAt')

4. arrayUnion('updatedAt')

正解: 2. orderBy('updatedAt', 'asc')

ヒント

並び替えはorderByです。

解説

orderByは指定フィールドで結果を並び替えます。whereと組み合わせる場合はインデックスが必要になることがあります。

061 FSQ-167

クエリ

初級

日替わり

priorityで昇順に並べて取得したい。Firestoreクエリで使うものは？

```
query(collection(db, 'orders'), orderBy('priority', 'asc'));
```

1. where('priority', 'asc')

2. limit('priority')

OK 3. orderBy('priority', 'asc')

4. arrayUnion('priority')

正解: 3. orderBy('priority', 'asc')

ヒント

並び替えはorderByです。

解説

orderByは指定フィールドで結果を並び替えます。whereと組み合わせる場合はインデックスが必要になることがあります。

062 FSQ-168

クエリ

中級

日替わり

orderMethodで昇順に並べて取得したい。Firestoreクエリで使うものは？

```
query(collection(db, 'orders'), orderBy('orderMethod', 'asc'));
```

1. where('orderMethod', 'asc')

2. limit('orderMethod')

3. arrayUnion('orderMethod')

OK 4. orderBy('orderMethod', 'asc')

正解: 4. orderBy('orderMethod', 'asc')

ヒント

並び替えはorderByです。

解説

orderByは指定フィールドで結果を並び替えます。whereと組み合わせる場合はインデックスが必要になることがあります。

063 FSQ-169

クエリ

中級

日替わり

answeredAtで昇順に並べて取得したい。Firestoreクエリで使うものは？

```
query(collection(db, 'orders'), orderBy('answeredAt', 'asc'));
```

OK 1. orderBy('answeredAt', 'asc')

2. where('answeredAt', 'asc')

3. limit('answeredAt')

4. arrayUnion('answeredAt')

正解: 1. orderBy('answeredAt', 'asc')

ヒント

並び替えはorderByです。

解説

orderByは指定フィールドで結果を並び替えます。whereと組み合わせる場合はインデックスが必要になることがあります。

064 FSQ-170

クエリ

応用

日替わり

categoryで昇順に並べて取得したい。Firestoreクエリで使うものは？

```
query(collection(db, 'orders'), orderBy('category', 'asc'));
```

1. where('category', 'asc')

OK 2. orderBy('category', 'asc')

3. limit('category')

4. arrayUnion('category')

正解: 2. orderBy('category', 'asc')

ヒント

並び替えはorderByです。

解説

orderByは指定フィールドで結果を並び替えます。whereと組み合わせる場合はインデックスが必要になることがあります。

065 FSQ-179

クエリ

中級

日替わり

FirestoreクエリAPI「startAfter」の用途として最も近いものは？

1. ドキュメントを物理削除する

2. サーバー時刻を保存する

OK 3. 指定したドキュメントの次から取得する

4. 配列に値を追加する

正解: 3. 指定したドキュメントの次から取得する

ヒント

クエリ結果の範囲、並び順、対象を調整するAPIです。

解説

startAfterはFirestoreクエリを組み立てるためのAPIです。一覧表示、ページング、横断検索などで使います。

066 FSQ-180

クエリ

応用

日替わり

FirestoreクエリAPI「startAt」の用途として最も近いものは？

1. ドキュメントを物理削除する

2. サーバー時刻を保存する

3. 配列に値を追加する

OK 4. 指定した位置から取得を始める

正解: 4. 指定した位置から取得を始める

ヒント

クエリ結果の範囲、並び順、対象を調整するAPIです。

解説

startAtはFirestoreクエリを組み立てるためのAPIです。一覧表示、ページング、横断検索などで使います。

067 FSQ-181

クエリ

初級

日替わり

FirestoreクエリAPI「endBefore」の用途として最も近いものは？

OK 1. 指定した位置の手前まで取得する

2. ドキュメントを物理削除する

3. サーバー時刻を保存する

4. 配列に値を追加する

正解: 1. 指定した位置の手前まで取得する

ヒント

クエリ結果の範囲、並び順、対象を調整するAPIです。

解説

endBeforeはFirestoreクエリを組み立てるためのAPIです。一覧表示、ページング、横断検索などで使います。

068 FSQ-182

クエリ

初級

日替わり

FirestoreクエリAPI「endAt」の用途として最も近いものは？

1. ドキュメントを物理削除する

OK 2. 指定した位置まで取得する

3. サーバー時刻を保存する

4. 配列に値を追加する

正解: 2. 指定した位置まで取得する

ヒント

クエリ結果の範囲、並び順、対象を調整するAPIです。

解説

endAtはFirestoreクエリを組み立てるためのAPIです。一覧表示、ページング、横断検索などで使います。

069 FSQ-183

クエリ

中級

日替わり

FirestoreクエリAPI「limitToLast」の用途として最も近いものは？

1. ドキュメントを物理削除する

2. サーバー時刻を保存する

OK 3. 並び順の末尾側から件数を制限する

4. 配列に値を追加する

正解: 3. 並び順の末尾側から件数を制限する

ヒント

クエリ結果の範囲、並び順、対象を調整するAPIです。

解説

limitToLastはFirestoreクエリを組み立てるためのAPIです。一覧表示、ページング、横断検索などで使います。

070 FSQ-184

クエリ

中級

日替わり

FirestoreクエリAPI「orderBy」の用途として最も近いものは？

1. ドキュメントを物理削除する

2. サーバー時刻を保存する

3. 配列に値を追加する

OK 4. 指定フィールドで並び替える

正解: 4. 指定フィールドで並び替える

ヒント

クエリ結果の範囲、並び順、対象を調整するAPIです。

解説

orderByはFirestoreクエリを組み立てるためのAPIです。一覧表示、ページング、横断検索などで使います。

071 FSQ-185

クエリ

応用

日替わり

FirestoreクエリAPI「where」の用途として最も近いものは？

OK 1. フィールド条件で絞り込む

2. ドキュメントを物理削除する

3. サーバー時刻を保存する

4. 配列に値を追加する

正解: 1. フィールド条件で絞り込む

ヒント

クエリ結果の範囲、並び順、対象を調整するAPIです。

解説

whereはFirestoreクエリを組み立てるためのAPIです。一覧表示、ページング、横断検索などで使います。

072 FSQ-186

クエリ

初級

日替わり

FirestoreクエリAPI「collectionGroup」の用途として最も近いものは？

1. ドキュメントを物理削除する

OK 2. 同名サブコレクションを横断検索する

3. サーバー時刻を保存する

4. 配列に値を追加する

正解: 2. 同名サブコレクションを横断検索する

ヒント

クエリ結果の範囲、並び順、対象を調整するAPIです。

解説

collectionGroupはFirestoreクエリを組み立てるためのAPIです。一覧表示、ページング、横断検索などで使います。

073 FSQ-203

クエリ

中級

日替わり

複数の親ドキュメント配下にある「items」サブコレクションを横断検索したい。使うものは？

```
collectionGroup(db, 'items')
```

1. docGroup

2. arrayUnion

OK 3. collectionGroup

4. serverTimestamp

正解: 3. collectionGroup

ヒント

同じ名前のサブコレクションを横断します。

解説

collectionGroupを使うと、階層の異なる場所にある同名サブコレクションをまとめて検索できます。

074 FSQ-204

クエリ

中級

daily

複数の親ドキュメント配下にある「answers」サブコレクションを横断検索したい。使うものは？

```
collectionGroup(db, 'answers')
```

1. docGroup

2. arrayUnion

3. serverTimestamp

OK 4. collectionGroup

正解: 4. collectionGroup

ヒント

同じ名前のサブコレクションを横断します。

解説

collectionGroupを使うと、階層の異なる場所にある同名サブコレクションをまとめて検索できます。

075 FSQ-205

クエリ

応用

日替わり

複数の親ドキュメント配下にある「messages」サブコレクションを横断検索したい。使うものは？

```
collectionGroup(db, 'messages')
```

OK 1. collectionGroup

2. docGroup

3. arrayUnion

4. serverTimestamp

正解: 1. collectionGroup

ヒント

同じ名前のサブコレクションを横断します。

解説

collectionGroupを使うと、階層の異なる場所にある同名サブコレクションをまとめて検索できます。

076 FSQ-206

クエリ

初級

日替わり

複数の親ドキュメント配下にある「logs」サブコレクションを横断検索したい。使うものは？

```
collectionGroup(db, 'logs')
```

1. docGroup

OK 2. collectionGroup

3. arrayUnion

4. serverTimestamp

正解: 2. collectionGroup

ヒント

同じ名前のサブコレクションを横断します。

解説

collectionGroupを使うと、階層の異なる場所にある同名サブコレクションをまとめて検索できます。

077 FSQ-207

クエリ

初級

日替わり

複数の親ドキュメント配下にある「comments」サブコレクションを横断検索したい。使うものは？

```
collectionGroup(db, 'comments')
```

1. docGroup

2. arrayUnion

OK 3. collectionGroup

4. serverTimestamp

正解: 3. collectionGroup

ヒント

同じ名前のサブコレクションを横断します。

解説

collectionGroupを使うと、階層の異なる場所にある同名サブコレクションをまとめて検索できます。

078 FSQ-208

クエリ

中級

日替わり

複数の親ドキュメント配下にある「scores」サブコレクションを横断検索したい。使うものは？

```
collectionGroup(db, 'scores')
```

1. docGroup

2. arrayUnion

3. serverTimestamp

OK 4. collectionGroup

正解: 4. collectionGroup

ヒント

同じ名前のサブコレクションを横断します。

解説

collectionGroupを使うと、階層の異なる場所にある同名サブコレクションをまとめて検索できます。

079 FSQ-209

クエリ

中級

日替わり

複数の親ドキュメント配下にある「tasks」サブコレクションを横断検索したい。使うものは？

```
collectionGroup(db, 'tasks')
```

OK 1. collectionGroup

2. docGroup

3. arrayUnion

4. serverTimestamp

正解: 1. collectionGroup

ヒント

同じ名前のサブコレクションを横断します。

解説

collectionGroupを使うと、階層の異なる場所にある同名サブコレクションをまとめて検索できます。

080 FSQ-210

クエリ

応用

日替わり

複数の親ドキュメント配下にある「events」サブコレクションを横断検索したい。使うものは？

```
collectionGroup(db, 'events')
```

1. docGroup

OK 2. collectionGroup

3. arrayUnion

4. serverTimestamp

正解: 2. collectionGroup

ヒント

同じ名前のサブコレクションを横断します。

解説

collectionGroupを使うと、階層の異なる場所にある同名サブコレクションをまとめて検索できます。

インデックス

10問 / 正解番号・正解テキスト・ヒント・解説付き

081 FSQ-047

インデックス

初級

全範囲

whereで絞り込み、statusでorderByしたクエリがFirestoreで失敗した。必要になることが多い設定は？

```
query(collection(db, 'orders'), where('status', '==', 'waiting'), orderBy('status'));
```

1. Storageバケット

2. CSSリセット

OK 3. 複合インデックス

4. HTMLのmetaタグ

正解: 3. 複合インデックス

ヒント

Firestoreのエラーに作成リンクが出ることがあります。

解説

Firestoreでは複数条件や並び替えを組み合わせるクエリで複合インデックスが必要になる場合があります。

082 FSQ-048

インデックス

中級

全範囲

whereで絞り込み、tableでorderByしたクエリがFirestoreで失敗した。必要になることが多い設定は？

```
query(collection(db, 'orders'), where('status', '==', 'waiting'), orderBy('table'));
```

1. Storageバケット

2. CSSリセット

3. HTMLのmetaタグ

OK 4. 複合インデックス

正解: 4. 複合インデックス

ヒント

Firestoreのエラーに作成リンクが出ることがあります。

解説

Firestoreでは複数条件や並び替えを組み合わせるクエリで複合インデックスが必要になる場合があります。

083 FSQ-049

インデックス

中級

全範囲

whereで絞り込み、guestCountでorderByしたクエリがFirestoreで失敗した。必要になることが多い設定は？

```
query(collection(db, 'orders'), where('status', '==', 'waiting'), orderBy('guestCount'));
```

OK 1. 複合インデックス

2. Storageバケット

3. CSSリセット

4. HTMLのmetaタグ

正解: 1. 複合インデックス

ヒント

Firestoreのエラーに作成リンクが出ることがあります。

解説

Firestoreでは複数条件や並び替えを組み合わせるクエリで複合インデックスが必要になる場合があります。

084 FSQ-050

インデックス

応用

全範囲

whereで絞り込み、staffCodeでorderByしたクエリがFirestoreで失敗した。必要になることが多い設定は？

```
query(collection(db, 'orders'), where('status', '==', 'waiting'), orderBy('staffCode'));
```

1. Storageバケット

OK 2. 複合インデックス

3. CSSリセット

4. HTMLのmetaタグ

正解: 2. 複合インデックス

ヒント

Firestoreのエラーに作成リンクが出ることがあります。

解説

Firestoreでは複数条件や並び替えを組み合わせるクエリで複合インデックスが必要になる場合があります。

085 FSQ-051

インデックス

初級

全範囲

whereで絞り込み、createdAtでorderByしたクエリがFirestoreで失敗した。必要になることが多い設定は？

```
query(collection(db, 'orders'), where('status', '==', 'waiting'), orderBy('createdAt'));
```

1. Storageバケット

2. CSSリセット

OK 3. 複合インデックス

4. HTMLのmetaタグ

正解: 3. 複合インデックス

ヒント

Firestoreのエラーに作成リンクが出ることがあります。

解説

Firestoreでは複数条件や並び替えを組み合わせるクエリで複合インデックスが必要になる場合があります。

086 FSQ-052

インデックス

初級

全範囲

whereで絞り込み、updatedAtでorderByしたクエリがFirestoreで失敗した。必要になることが多い設定は？

```
query(collection(db, 'orders'), where('status', '==', 'waiting'), orderBy('updatedAt'));
```

1. Storageバケット

2. CSSリセット

3. HTMLのmetaタグ

OK 4. 複合インデックス

正解: 4. 複合インデックス

ヒント

Firestoreのエラーに作成リンクが出ることがあります。

解説

Firestoreでは複数条件や並び替えを組み合わせるクエリで複合インデックスが必要になる場合があります。

087 FSQ-053

インデックス

中級

daily

whereで絞り込み、priorityでorderByしたクエリがFirestoreで失敗した。必要になることが多い設定は？

```
query(collection(db, 'orders'), where('status', '==', 'waiting'), orderBy('priority'));
```

OK 1. 複合インデックス

2. HTMLのmetaタグ

3. Storageバケット

4. CSSリセット

正解: 1. 複合インデックス

ヒント

Firestoreのエラーに作成リンクが出ることがあります。

解説

Firestoreでは複数条件や並び替えを組み合わせるクエリで複合インデックスが必要になる場合があります。

088 FSQ-054

インデックス

中級

全範囲

whereで絞り込み、orderByでorderByしたクエリがFirestoreで失敗した。必要になることが多い設定は？

```
query(collection(db, 'orders'), where('status', '==', 'waiting'), orderBy('orderMethod'));
```

1. Storageバケット

OK 2. 複合インデックス

3. CSSリセット

4. HTMLのmetaタグ

正解: 2. 複合インデックス

ヒント

Firestoreのエラーに作成リンクが出ることがあります。

解説

Firestoreでは複数条件や並び替えを組み合わせるクエリで複合インデックスが必要になる場合があります。

089 FSQ-055

インデックス

応用

全範囲

whereで絞り込み、answeredAtでorderByしたクエリがFirestoreで失敗した。必要になることが多い設定は？

```
query(collection(db, 'orders'), where('status', '==', 'waiting'), orderBy('answeredAt'));
```

1. Storageバケット

2. CSSリセット

OK 3. 複合インデックス

4. HTMLのmetaタグ

正解: 3. 複合インデックス

ヒント

Firestoreのエラーに作成リンクが出ることがあります。

解説

Firestoreでは複数条件や並び替えを組み合わせるクエリで複合インデックスが必要になる場合があります。

090 FSQ-056

インデックス

初級

全範囲

whereで絞り込み、categoryでorderByしたクエリがFirestoreで失敗した。必要になることが多い設定は？

```
query(collection(db, 'orders'), where('status', '==', 'waiting'), orderBy('category'));
```

1. Storageバケット

2. CSSリセット

3. HTMLのmetaタグ

OK 4. 複合インデックス

正解: 4. 複合インデックス

ヒント

Firestoreのエラーに作成リンクが出ることがあります。

解説

Firestoreでは複数条件や並び替えを組み合わせるクエリで複合インデックスが必要になる場合があります。

データ設計

22問 / 正解番号・正解テキスト・ヒント・解説付き

091 FSQ-057

データ設計

初級

全範囲

注文状態「waiting」をFirestoreで扱いやすく保存したい。自然なフィールド設計は？

```
{ status: 'waiting' }
```

OK 1. statusフィールドに状態値として保存する

2. 背景色だけを保存する

3. HTMLのボタン文言だけで判断する

4. ドキュメントIDに毎回埋め込む

正解: 1. statusフィールドに状態値として保存する

ヒント

状態は表示ではなくデータとして保存します。

解説

statusをフィールドとして保存すると、whereで絞り込み、KDS表示、履歴集計などに使えます。

092 FSQ-058

データ設計

中級

全範囲

注文状態「cooking」をFirestoreで扱いやすく保存したい。自然なフィールド設計は？

```
{ status: 'cooking' }
```

1. 背景色だけを保存する

OK 2. statusフィールドに状態値として保存する

3. HTMLのボタン文言だけで判断する

4. ドキュメントIDに毎回埋め込む

正解: 2. statusフィールドに状態値として保存する

ヒント

状態は表示ではなくデータとして保存します。

解説

statusをフィールドとして保存すると、whereで絞り込み、KDS表示、履歴集計などに使えます。

093 FSQ-059

データ設計

中級

全範囲

注文状態「ready」をFirestoreで扱いやすく保存したい。自然なフィールド設計は？

```
{ status: 'ready' }
```

1. 背景色だけを保存する

2. HTMLのボタン文言だけで判断する

OK 3. statusフィールドに状態値として保存する

4. ドキュメントIDに毎回埋め込む

正解: 3. statusフィールドに状態値として保存する

ヒント

状態は表示ではなくデータとして保存します。

解説

statusをフィールドとして保存すると、whereで絞り込み、KDS表示、履歴集計などに使えます。

094 FSQ-060

データ設計

応用

全範囲

注文状態「served」をFirestoreで扱いやすく保存したい。自然なフィールド設計は？

```
{ status: 'served' }
```

1. 背景色だけを保存する

2. HTMLのボタン文言だけで判断する

3. ドキュメントIDに毎回埋め込む

OK 4. statusフィールドに状態値として保存する

正解: 4. statusフィールドに状態値として保存する

ヒント

状態は表示ではなくデータとして保存します。

解説

statusをフィールドとして保存すると、whereで絞り込み、KDS表示、履歴集計などに使えます。

095 FSQ-061

データ設計

初級

全範囲

注文状態「cancelled」をFirestoreで扱いやすく保存したい。自然なフィールド設計は？

```
{ status: 'cancelled' }
```

OK 1. statusフィールドに状態値として保存する

2. 背景色だけを保存する

3. HTMLのボタン文言だけで判断する

4. ドキュメントIDに毎回埋め込む

正解: 1. statusフィールドに状態値として保存する

ヒント

状態は表示ではなくデータとして保存します。

解説

statusをフィールドとして保存すると、whereで絞り込み、KDS表示、履歴集計などに使えます。

096 FSQ-062

データ設計

初級

全範囲

注文状態「hidden」をFirestoreで扱いやすく保存したい。自然なフィールド設計は？

```
{ status: 'hidden' }
```

1. 背景色だけを保存する

OK 2. statusフィールドに状態値として保存する

3. HTMLのボタン文言だけで判断する

4. ドキュメントIDに毎回埋め込む

正解: 2. statusフィールドに状態値として保存する

ヒント

状態は表示ではなくデータとして保存します。

解説

statusをフィールドとして保存すると、whereで絞り込み、KDS表示、履歴集計などに使えます。

097 FSQ-063

データ設計

中級

daily

注文状態「archived」をFirestoreで扱いやすく保存したい。自然なフィールド設計は？

```
{ status: 'archived' }
```

1. 背景色だけを保存する

OK 2. statusフィールドに状態値として保存する

3. HTMLのボタン文言だけで判断する

4. ドキュメントIDに毎回埋め込む

正解: 2. statusフィールドに状態値として保存する

ヒント

状態は表示ではなくデータとして保存します。

解説

statusをフィールドとして保存すると、whereで絞り込み、KDS表示、履歴集計などに使えます。

098 FSQ-064

データ設計

中級

全範囲

注文状態「delayed」をFirestoreで扱いやすく保存したい。自然なフィールド設計は？

```
{ status: 'delayed' }
```

1. 背景色だけを保存する

2. HTMLのボタン文言だけで判断する

3. ドキュメントIDに毎回埋め込む

OK 4. statusフィールドに状態値として保存する

正解: 4. statusフィールドに状態値として保存する

ヒント

状態は表示ではなくデータとして保存します。

解説

statusをフィールドとして保存すると、whereで絞り込み、KDS表示、履歴集計などに使えます。

099 FSQ-187

データ設計

初級

全範囲

Firestoreに保存できるデータ型「string」について正しい考え方は？

1. すべてHTML文字列に変換しないと保存できない

2. CSSファイルにだけ保存できる

OK 3. 文字列として保存し、完全一致や範囲条件に使える

4. Security Rulesでは一切判定できない

正解: 3. 文字列として保存し、完全一致や範囲条件に使える

ヒント

Firestoreは複数のデータ型を扱えます。

解説

Firestoreでは文字列、数値、真偽値、Timestamp、Map、Arrayなどを保存できます。用途に合う型で保存すると検索や集計がしやすくなります。

100 FSQ-188

データ設計

中級

全範囲

Firestoreに保存できるデータ型「number」について正しい考え方は？

1. すべてHTML文字列に変換しないと保存できない

2. CSSファイルにだけ保存できる

3. Security Rulesでは一切判定できない

OK 4. 数値として保存し、大小比較や加算に使える

正解: 4. 数値として保存し、大小比較や加算に使える

ヒント

Firestoreは複数のデータ型を扱えます。

解説

Firestoreでは文字列、数値、真偽値、Timestamp、Map、Arrayなどを保存できます。用途に合う型で保存すると検索や集計がしやすくなります。

101 FSQ-189

データ設計

中級

全範囲

Firestoreに保存できるデータ型「boolean」について正しい考え方は？

OK 1. true/falseとして状態判定に使える

2. すべてHTML文字列に変換しないと保存できない

3. CSSファイルにだけ保存できる

4. Security Rulesでは一切判定できない

正解: 1. true/falseとして状態判定に使える

ヒント

Firestoreは複数のデータ型を扱えます。

解説

Firestoreでは文字列、数値、真偽値、Timestamp、Map、Arrayなどを保存できます。用途に合う型で保存すると検索や集計がしやすくなります。

102 FSQ-190

データ設計

応用

全範囲

Firestoreに保存できるデータ型「timestamp」について正しい考え方は？

1. すべてHTML文字列に変換しないと保存できない

OK 2. 日時として並び替えや期間条件に使える

3. CSSファイルにだけ保存できる

4. Security Rulesでは一切判定できない

正解: 2. 日時として並び替えや期間条件に使える

ヒント

Firestoreは複数のデータ型を扱えます。

解説

Firestoreでは文字列、数値、真偽値、Timestamp、Map、Arrayなどを保存できます。用途に合う型で保存すると検索や集計がしやすくなります。

103 FSQ-191

データ設計

初級

全範囲

Firestoreに保存できるデータ型「map」について正しい考え方は？

1. すべてHTML文字列に変換しないと保存できない

2. CSSファイルにだけ保存できる

OK 3. 関連する複数フィールドを入れ子でまとめられる

4. Security Rulesでは一切判定できない

正解: 3. 関連する複数フィールドを入れ子でまとめられる

ヒント

Firestoreは複数のデータ型を扱えます。

解説

Firestoreでは文字列、数値、真偽値、Timestamp、Map、Arrayなどを保存できます。用途に合う型で保存すると検索や集計がしやすくなります。

104 FSQ-192

データ設計

初級

全範囲

Firestoreに保存できるデータ型「array」について正しい考え方は？

1. すべてHTML文字列に変換しないと保存できない

2. CSSファイルにだけ保存できる

3. Security Rulesでは一切判定できない

OK 4. 複数值を1フィールドに保持できる

正解: 4. 複数值を1フィールドに保持できる

ヒント

Firestoreは複数のデータ型を扱えます。

解説

Firestoreでは文字列、数値、真偽値、Timestamp、Map、Arrayなどを保存できます。用途に合う型で保存すると検索や集計がしやすくなります。

105 FSQ-193

データ設計

中級

全範囲

Firestoreに保存できるデータ型「null」について正しい考え方は？

OK 1. 値がない状態を明示できる

2. すべてHTML文字列に変換しないと保存できない

3. CSSファイルにだけ保存できる

4. Security Rulesでは一切判定できない

正解: 1. 値がない状態を明示できる

ヒント

Firestoreは複数のデータ型を扱えます。

解説

Firestoreでは文字列、数値、真偽値、Timestamp、Map、Arrayなどを保存できます。用途に合う型で保存すると検索や集計がしやすくなります。

106 FSQ-194

データ設計

中級

全範囲

Firestoreに保存できるデータ型「reference」について正しい考え方は？

1. すべてHTML文字列に変換しないと保存できない

OK 2. 別ドキュメントへの参照値として保存できる

3. CSSファイルにだけ保存できる

4. Security Rulesでは一切判定できない

正解: 2. 別ドキュメントへの参照値として保存できる

ヒント

Firestoreは複数のデータ型を扱えます。

解説

Firestoreでは文字列、数値、真偽値、Timestamp、Map、Arrayなどを保存できます。用途に合う型で保存すると検索や集計がしやすくなります。

107 FSQ-235

データ設計

応用

全範囲

注文IDをFirestoreのドキュメントIDとして使う場合に意識することは？

1. 必ず日本語だけにする

2. 毎回空文字にする

OK 3. 一意性、推測しやすさ、変更される可能性を考える

4. CSSクラス名と同じにする

正解: 3. 一意性、推測しやすさ、変更される可能性を考える

ヒント

IDはドキュメントの場所そのものになります。

解説

ドキュメントIDはパスに含まれるため、一意性や変更可能性、権限判定との相性を考えて設計します。

108 FSQ-236

データ設計

初級

全範囲

ユーザーIDをFirestoreのドキュメントIDとして使う場合に意識することは？

1. 必ず日本語だけにする

2. 毎回空文字にする

3. CSSクラス名と同じにする

OK 4. 一意性、推測しやすさ、変更される可能性を考える

正解: 4. 一意性、推測しやすさ、変更される可能性を考える

ヒント

IDはドキュメントの場所そのものになります。

解説

ドキュメントIDはパスに含まれるため、一意性や変更可能性、権限判定との相性を考えて設計します。

109 FSQ-237

データ設計

初級

全範囲

商品IDをFirestoreのドキュメントIDとして使う場合に意識することは？

OK 1. 一意性、推測しやすさ、変更される可能性を考える

2. 必ず日本語だけにする

3. 毎回空文字にする

4. CSSクラス名と同じにする

正解: 1. 一意性、推測しやすさ、変更される可能性を考える

ヒント

IDはドキュメントの場所そのものになります。

解説

ドキュメントIDはパスに含まれるため、一意性や変更可能性、権限判定との相性を考えて設計します。

110 FSQ-238

データ設計

中級

全範囲

問題IDをFirestoreのドキュメントIDとして使う場合に意識することは？

1. 必ず日本語だけにする

OK 2. 一意性、推測しやすさ、変更される可能性を考える

3. 毎回空文字にする

4. CSSクラス名と同じにする

正解: 2. 一意性、推測しやすさ、変更される可能性を考える

ヒント

IDはドキュメントの場所そのものになります。

解説

ドキュメントIDはパスに含まれるため、一意性や変更可能性、権限判定との相性を考えて設計します。

111 FSQ-239

データ設計

中級

全範囲

回答IDをFirestoreのドキュメントIDとして使う場合に意識することは？

1. 必ず日本語だけにする

2. 毎回空文字にする

OK 3. 一意性、推測しやすさ、変更される可能性を考える

4. CSSクラス名と同じにする

正解: 3. 一意性、推測しやすさ、変更される可能性を考える

ヒント

IDはドキュメントの場所そのものになります。

解説

ドキュメントIDはパスに含まれるため、一意性や変更可能性、権限判定との相性を考えて設計します。

112 FSQ-240

データ設計

応用

全範囲

店舗IDをFirestoreのドキュメントIDとして使う場合に意識することは？

1. 必ず日本語だけにする

2. 毎回空文字にする

3. CSSクラス名と同じにする

OK 4. 一意性、推測しやすさ、変更される可能性を考える

正解: 4. 一意性、推測しやすさ、変更される可能性を考える

ヒント

IDはドキュメントの場所そのものになります。

解説

ドキュメントIDはパスに含まれるため、一意性や変更可能性、権限判定との相性を考えて設計します。

タイムスタンプ

8問 / 正解番号・正解テキスト・ヒント・解説付き

113 FSQ-065

タイムスタンプ

応用

全範囲

createdAtを端末時計ではなくFirestore側の時刻で保存したい。使う値は？

```
{ createdAt: serverTimestamp() }
```

OK 1. serverTimestamp()

2. Date.now()だけ

3. Math.random()

4. document.title

正解: 1. serverTimestamp()

ヒント

サーバー側で確定する時刻です。

解説

serverTimestamp()を使うとFirestoreサーバー基準の時刻を保存できます。複数端末で時刻ズレがある場合にも扱いやすくなります。

114 FSQ-066

タイムスタンプ

初級

全範囲

updatedAtを端末時計ではなくFirestore側の時刻で保存したい。使う値は？

```
{ updatedAt: serverTimestamp() }
```

1. Date.now()だけ

OK 2. serverTimestamp()

3. Math.random()

4. document.title

正解: 2. serverTimestamp()

ヒント

サーバー側で確定する時刻です。

解説

serverTimestamp()を使うとFirestoreサーバー基準の時刻を保存できます。複数端末で時刻ズレがある場合にも扱いやすくなります。

115 FSQ-067

タイムスタンプ

初級

全範囲

startedAtを端末時計ではなくFirestore側の時刻で保存したい。使う値は？

```
{ startedAt: serverTimestamp() }
```

1. Date.now()だけ

2. Math.random()

OK 3. serverTimestamp()

4. document.title

正解: 3. serverTimestamp()

ヒント

サーバー側で確定する時刻です。

解説

serverTimestamp()を使うとFirestoreサーバー基準の時刻を保存できます。複数端末で時刻ズレがある場合にも扱いやすくなります。

116 FSQ-068

タイムスタンプ

中級

全範囲

completedAtを端末時計ではなくFirestore側の時刻で保存したい。使う値は？

```
{ completedAt: serverTimestamp() }
```

1. Date.now()だけ

2. Math.random()

3. document.title

OK 4. serverTimestamp()

正解: 4. serverTimestamp()

ヒント

サーバー側で確定する時刻です。

解説

serverTimestamp()を使うとFirestoreサーバー基準の時刻を保存できます。複数端末で時刻ズレがある場合にも扱いやすくなります。

117 FSQ-069

タイムスタンプ

中級

全範囲

cancelledAtを端末時計ではなくFirestore側の時刻で保存したい。使う値は？

```
{ cancelledAt: serverTimestamp() }
```

OK 1. serverTimestamp()

2. Date.now()だけ

3. Math.random()

4. document.title

正解: 1. serverTimestamp()

ヒント

サーバー側で確定する時刻です。

解説

serverTimestamp()を使うとFirestoreサーバー基準の時刻を保存できます。複数端末で時刻ズレがある場合にも扱いやすくなります。

118 FSQ-070

タイムスタンプ

応用

全範囲

answeredAtを端末時計ではなくFirestore側の時刻で保存したい。使う値は？

```
{ answeredAt: serverTimestamp() }
```

1. Date.now()だけ

OK 2. serverTimestamp()

3. Math.random()

4. document.title

正解: 2. serverTimestamp()

ヒント

サーバー側で確定する時刻です。

解説

serverTimestamp()を使うとFirestoreサーバー基準の時刻を保存できます。複数端末で時刻ズレがある場合にも扱いやすくなります。

119 FSQ-071

タイムスタンプ

初級

全範囲

hiddenAtを端末時計ではなくFirestore側の時刻で保存したい。使う値は？

```
{ hiddenAt: serverTimestamp() }
```

1. Date.now()だけ

2. Math.random()

OK 3. serverTimestamp()

4. document.title

正解: 3. serverTimestamp()

ヒント

サーバー側で確定する時刻です。

解説

serverTimestamp()を使うとFirestoreサーバー基準の時刻を保存できます。複数端末で時刻ズレがある場合にも扱いやすくなります。

120 FSQ-072

タイムスタンプ

初級

全範囲

lastLoginAtを端末時計ではなくFirestore側の時刻で保存したい。使う値は？

```
{ lastLoginAt: serverTimestamp() }
```

1. Date.now()だけ

2. Math.random()

3. document.title

OK 4. serverTimestamp()

正解: 4. serverTimestamp()

ヒント

サーバー側で確定する時刻です。

解説

serverTimestamp()を使うとFirestoreサーバー基準の時刻を保存できます。複数端末で時刻ズレがある場合にも扱いやすくなります。

リアルタイム監視

8問 / 正解番号・正解テキスト・ヒント・解説付き

121 FSQ-073

リアルタイム監視

中級

全範囲

KDS表示をFirestoreの変更に合わせて自動更新したい。向いているAPIは？

```
const unsubscribe = onSnapshot(q, snapshot => { ... });
```

OK 1. onSnapshot

2. getDocs

3. deleteDoc

4. parseInt

正解: 1. onSnapshot

ヒント

変更を購読するAPIです。

解説

onSnapshotは初回取得に加えて、その後の追加・変更・削除をリアルタイムに受け取れます。

122 FSQ-074

リアルタイム監視

中級

全範囲

注文一覧をFirestoreの変更に合わせて自動更新したい。向いているAPIは？

```
const unsubscribe = onSnapshot(q, snapshot => { ... });
```

1. getDocs

OK 2. onSnapshot

3. deleteDoc

4. parseInt

正解: 2. onSnapshot

ヒント

変更を購読するAPIです。

解説

onSnapshotは初回取得に加えて、その後の追加・変更・削除をリアルタイムに受け取れます。

123 FSQ-075

リアルタイム監視

応用

全範囲

学習ログ一覧をFirestoreの変更に合わせて自動更新したい。向いているAPIは？

```
const unsubscribe = onSnapshot(q, snapshot => { ... });
```

1. getDocs

2. deleteDoc

OK 3. onSnapshot

4. parseInt

正解: 3. onSnapshot

ヒント

変更を購読するAPIです。

解説

onSnapshotは初回取得に加えて、その後の追加・変更・削除をリアルタイムに受け取れます。

124 FSQ-076

リアルタイム監視

初級

daily

提供待ち一覧をFirestoreの変更に合わせて自動更新したい。向いているAPIは？

```
const unsubscribe = onSnapshot(q, snapshot => { ... });
```

1. parseInt

OK 2. onSnapshot

3. getDocs

4. deleteDoc

正解: 2. onSnapshot

ヒント

変更を購読するAPIです。

解説

onSnapshotは初回取得に加えて、その後の追加・変更・削除をリアルタイムに受け取れます。

125 FSQ-077

リアルタイム監視

初級

全範囲

調理中一覧をFirestoreの変更に合わせて自動更新したい。向いているAPIは？

```
const unsubscribe = onSnapshot(q, snapshot => { ... });
```

OK 1. onSnapshot

2. getDocs

3. deleteDoc

4. parseInt

正解: 1. onSnapshot

ヒント

変更を購読するAPIです。

解説

onSnapshotは初回取得に加えて、その後の追加・変更・削除をリアルタイムに受け取れます。

126 FSQ-078

リアルタイム監視

中級

全範囲

ユーザー別進捗をFirestoreの変更に合わせて自動更新したい。向いているAPIは？

```
const unsubscribe = onSnapshot(q, snapshot => { ... });
```

1. getDocs

OK 2. onSnapshot

3. deleteDoc

4. parseInt

正解: 2. onSnapshot

ヒント

変更を購読するAPIです。

解説

onSnapshotは初回取得に加えて、その後の追加・変更・削除をリアルタイムに受け取れます。

127 FSQ-079

リアルタイム監視

中級

全範囲

問題マスタをFirestoreの変更に合わせて自動更新したい。向いているAPIは？

```
const unsubscribe = onSnapshot(q, snapshot => { ... });
```

1. getDocs

2. deleteDoc

OK 3. onSnapshot

4. parseInt

正解: 3. onSnapshot

ヒント

変更を購読するAPIです。

解説

onSnapshotは初回取得に加えて、その後の追加・変更・削除をリアルタイムに受け取れます。

128 FSQ-080

リアルタイム監視

応用

全範囲

設定変更をFirestoreの変更に合わせて自動更新したい。向いているAPIは？

```
const unsubscribe = onSnapshot(q, snapshot => { ... });
```

1. getDocs

2. deleteDoc

3. parseInt

OK 4. onSnapshot

正解: 4. onSnapshot

ヒント

変更を購読するAPIです。

解説

onSnapshotは初回取得に加えて、その後の追加・変更・削除をリアルタイムに受け取れます。

配列操作

16問 / 正解番号・正解テキスト・ヒント・解説付き

129 FSQ-081

配列操作

初級

全範囲

items配列へ値を追加し、重複追加を避けたい。Firestoreで使える操作は？

```
await updateDoc(ref, { items: arrayUnion(value) });
```

OK 1. arrayUnion

2. arrayRemove

3. serverTimestamp

4. orderBy

正解: 1. arrayUnion

ヒント

配列に値を追加する専用操作です。

解説

arrayUnionは配列に値を追加し、同じ値が既にある場合は重複追加しません。削除にはarrayRemoveを使います。

130 FSQ-082

配列操作

初級

全範囲

tags配列へ値を追加し、重複追加を避けたい。Firestoreで使える操作は？

```
await updateDoc(ref, { tags: arrayUnion(value) });
```

1. arrayRemove

OK 2. arrayUnion

3. serverTimestamp

4. orderBy

正解: 2. arrayUnion

ヒント

配列に値を追加する専用操作です。

解説

arrayUnionは配列に値を追加し、同じ値が既にある場合は重複追加しません。削除にはarrayRemoveを使います。

131 FSQ-083

配列操作

中級

全範囲

allergies配列へ値を追加し、重複追加を避けたい。Firestoreで使える操作は？

```
await updateDoc(ref, { allergies: arrayUnion(value) });
```

1. arrayRemove

2. serverTimestamp

OK 3. arrayUnion

4. orderBy

正解: 3. arrayUnion

ヒント

配列に値を追加する専用操作です。

解説

arrayUnionは配列に値を追加し、同じ値が既にある場合は重複追加しません。削除にはarrayRemoveを使います。

132 FSQ-084

配列操作

中級

全範囲

selectedOptions配列へ値を追加し、重複追加を避けたい。Firestoreで使える操作は？

```
await updateDoc(ref, { selectedOptions: arrayUnion(value) });
```

1. arrayRemove

2. serverTimestamp

3. orderBy

OK 4. arrayUnion

正解: 4. arrayUnion

ヒント

配列に値を追加する専用操作です。

解説

arrayUnionは配列に値を追加し、同じ値が既にある場合は重複追加しません。削除にはarrayRemoveを使います。

133 FSQ-085

配列操作

応用

daily

wrongQuestionIds配列へ値を追加し、重複追加を避けたい。Firestoreで使える操作は？

```
await updateDoc(ref, { wrongQuestionIds: arrayUnion(value) });
```

1. arrayRemove

2. serverTimestamp

OK 3. arrayUnion

4. orderBy

正解: 3. arrayUnion

ヒント

配列に値を追加する専用操作です。

解説

arrayUnionは配列に値を追加し、同じ値が既にある場合は重複追加しません。削除にはarrayRemoveを使います。

134 FSQ-086

配列操作

初級

全範囲

completedItemIds配列へ値を追加し、重複追加を避けたい。Firestoreで使える操作は？

```
await updateDoc(ref, { completedItemIds: arrayUnion(value) });
```

1. arrayRemove

OK 2. arrayUnion

3. serverTimestamp

4. orderBy

正解: 2. arrayUnion

ヒント

配列に値を追加する専用操作です。

解説

arrayUnionは配列に値を追加し、同じ値が既にある場合は重複追加しません。削除にはarrayRemoveを使います。

135 FSQ-087

配列操作

初級

全範囲

staffCodes配列へ値を追加し、重複追加を避けたい。Firestoreで使える操作は？

```
await updateDoc(ref, { staffCodes: arrayUnion(value) });
```

1. arrayRemove

2. serverTimestamp

OK 3. arrayUnion

4. orderBy

正解: 3. arrayUnion

ヒント

配列に値を追加する専用操作です。

解説

arrayUnionは配列に値を追加し、同じ値が既にある場合は重複追加しません。削除にはarrayRemoveを使います。

136 FSQ-088

配列操作

中級

全範囲

menuIds配列へ値を追加し、重複追加を避けたい。Firestoreで使える操作は？

```
await updateDoc(ref, { menuIds: arrayUnion(value) });
```

1. arrayRemove

2. serverTimestamp

3. orderBy

OK 4. arrayUnion

正解: 4. arrayUnion

ヒント

配列に値を追加する専用操作です。

解説

arrayUnionは配列に値を追加し、同じ値が既にある場合は重複追加しません。削除にはarrayRemoveを使います。

137 FSQ-089

配列操作

中級

全範囲

items配列から特定の値を削除したい。Firestoreで使える操作は？

```
await updateDoc(ref, { items: arrayRemove(value) });
```

OK 1. arrayRemove

2. arrayUnion

3. limit

4. collection

正解: 1. arrayRemove

ヒント

配列から値を取り除く専用操作です。

解説

arrayRemoveは配列内の一致する値を削除します。商品明細のようにオブジェクト配列を細かく扱う場合は、配列再作成やサブコレクションも検討します。

138 FSQ-090

配列操作

応用

全範囲

tags配列から特定の値を削除したい。Firestoreで使える操作は？

```
await updateDoc(ref, { tags: arrayRemove(value) });
```

1. arrayUnion

OK 2. arrayRemove

3. limit

4. collection

正解: 2. arrayRemove

ヒント

配列から値を取り除く専用操作です。

解説

arrayRemoveは配列内の一致する値を削除します。商品明細のようにオブジェクト配列を細かく扱う場合は、配列再作成やサブコレクションも検討します。

139 FSQ-091

配列操作

初級

全範囲

allergies配列から特定の値を削除したい。Firestoreで使える操作は？

```
await updateDoc(ref, { allergies: arrayRemove(value) });
```

1. arrayUnion

2. limit

OK 3. arrayRemove

4. collection

正解: 3. arrayRemove

ヒント

配列から値を取り除く専用操作です。

解説

arrayRemoveは配列内の一致する値を削除します。商品明細のようにオブジェクト配列を細かく扱う場合は、配列再作成やサブコレクションも検討します。

140 FSQ-092

配列操作

初級

全範囲

selectedOptions配列から特定の値を削除したい。Firestoreで使える操作は？

```
await updateDoc(ref, { selectedOptions: arrayRemove(value) });
```

1. arrayUnion

2. limit

3. collection

OK 4. arrayRemove

正解: 4. arrayRemove

ヒント

配列から値を取り除く専用操作です。

解説

arrayRemoveは配列内の一致する値を削除します。商品明細のようにオブジェクト配列を細かく扱う場合は、配列再作成やサブコレクションも検討します。

141 FSQ-093

配列操作

中級

全範囲

wrongQuestionIds配列から特定の値を削除したい。Firestoreで使える操作は？

```
await updateDoc(ref, { wrongQuestionIds: arrayRemove(value) });
```

OK 1. arrayRemove

2. arrayUnion

3. limit

4. collection

正解: 1. arrayRemove

ヒント

配列から値を取り除く専用操作です。

解説

arrayRemoveは配列内の一致する値を削除します。商品明細のようにオブジェクト配列を細かく扱う場合は、配列再作成やサブコレクションも検討します。

142 FSQ-094

配列操作

中級

全範囲

completedItemIds配列から特定の値を削除したい。Firestoreで使える操作は？

```
await updateDoc(ref, { completedItemIds: arrayRemove(value) });
```

1. arrayUnion

OK 2. arrayRemove

3. limit

4. collection

正解: 2. arrayRemove

ヒント

配列から値を取り除く専用操作です。

解説

arrayRemoveは配列内の一致する値を削除します。商品明細のようにオブジェクト配列を細かく扱う場合は、配列再作成やサブコレクションも検討します。

143 FSQ-095

配列操作

応用

全範囲

staffCodes配列から特定の値を削除したい。Firestoreで使える操作は？

```
await updateDoc(ref, { staffCodes: arrayRemove(value) });
```

1. arrayUnion

2. limit

OK 3. arrayRemove

4. collection

正解: 3. arrayRemove

ヒント

配列から値を取り除く専用操作です。

解説

arrayRemoveは配列内の一致する値を削除します。商品明細のようにオブジェクト配列を細かく扱う場合は、配列再作成やサブコレクションも検討します。

144 FSQ-096

配列操作

初級

全範囲

menuIds配列から特定の値を削除したい。Firestoreで使える操作は？

```
await updateDoc(ref, { menuIds: arrayRemove(value) });
```

1. arrayUnion

2. limit

3. collection

OK 4. arrayRemove

正解: 4. arrayRemove

ヒント

配列から値を取り除く専用操作です。

解説

arrayRemoveは配列内の一致する値を削除します。商品明細のようにオブジェクト配列を細かく扱う場合は、配列再作成やサブコレクションも検討します。

サブコレクション

16問 / 正解番号・正解テキスト・ヒント・解説付き

145 FSQ-097

サブコレクション

初級

全範囲

ordersの1件に紐づく明細を多数保存し、明細単位で更新したい。検討しやすい設計は？

```
orders/{docId}/items/{itemId}
```

OK 1. サブコレクションに分ける

2. 全明細をCSSに保存する

3. 必ず1つの巨大配列にする

4. 毎回プロジェクトを作り直す

正解: 1. サブコレクションに分ける

ヒント

親ドキュメントの下に子コレクションを作る考え方です。

解説

サブコレクションにすると明細単位の追加、更新、削除、監視がしやすくなります。ただし読み取り回数やクエリ設計も考える必要があります。

146 FSQ-098

サブコレクション

中級

全範囲

menusの1件に紐づく明細を多数保存し、明細単位で更新したい。検討しやすい設計は？

```
menus/{docId}/items/{itemId}
```

1. 全明細をCSSに保存する

OK 2. サブコレクションに分ける

3. 必ず1つの巨大配列にする

4. 毎回プロジェクトを作り直す

正解: 2. サブコレクションに分ける

ヒント

親ドキュメントの下に子コレクションを作る考え方です。

解説

サブコレクションにすると明細単位の追加、更新、削除、監視がしやすくなります。ただし読み取り回数やクエリ設計も考える必要があります。

147 FSQ-099

サブコレクション

中級

全範囲

usersの1件に紐づく明細を多数保存し、明細単位で更新したい。検討しやすい設計は？

```
users/{docId}/items/{itemId}
```

1. 全明細をCSSに保存する

2. 必ず1つの巨大配列にする

OK 3. サブコレクションに分ける

4. 毎回プロジェクトを作り直す

正解: 3. サブコレクションに分ける

ヒント

親ドキュメントの下に子コレクションを作る考え方です。

解説

サブコレクションにすると明細単位の追加、更新、削除、監視がしやすくなります。ただし読み取り回数やクエリ設計も考える必要があります。

148 FSQ-100

サブコレクション

応用

全範囲

CookListの1件に紐づく明細を多数保存し、明細単位で更新したい。検討しやすい設計は？

```
CookList/{docId}/items/{itemId}
```

1. 全明細をCSSに保存する

2. 必ず1つの巨大配列にする

3. 毎回プロジェクトを作り直す

OK 4. サブコレクションに分ける

正解: 4. サブコレクションに分ける

ヒント

親ドキュメントの下に子コレクションを作る考え方です。

解説

サブコレクションにすると明細単位の追加、更新、削除、監視がしやすくなります。ただし読み取り回数やクエリ設計も考える必要があります。

149 FSQ-101

サブコレクション

初級

全範囲

StaffListの1件に紐づく明細を多数保存し、明細単位で更新したい。検討しやすい設計は？

```
StaffList/{docId}/items/{itemId}
```

OK 1. サブコレクションに分ける

2. 全明細をCSSに保存する

3. 必ず1つの巨大配列にする

4. 毎回プロジェクトを作り直す

正解: 1. サブコレクションに分ける

ヒント

親ドキュメントの下に子コレクションを作る考え方です。

解説

サブコレクションにすると明細単位の追加、更新、削除、監視がしやすくなります。ただし読み取り回数やクエリ設計も考える必要があります。

150 FSQ-102

サブコレクション

初級

全範囲

quizQuestionsの1件に紐づく明細を多数保存し、明細単位で更新したい。検討しやすい設計は？

```
quizQuestions/{docId}/items/{itemId}
```

1. 全明細をCSSに保存する

OK 2. サブコレクションに分ける

3. 必ず1つの巨大配列にする

4. 毎回プロジェクトを作り直す

正解: 2. サブコレクションに分ける

ヒント

親ドキュメントの下に子コレクションを作る考え方です。

解説

サブコレクションにすると明細単位の追加、更新、削除、監視がしやすくなります。ただし読み取り回数やクエリ設計も考える必要があります。

151 FSQ-103

サブコレクション

中級

全範囲

learningLogsの1件に紐づく明細を多数保存し、明細単位で更新したい。検討しやすい設計は？

```
learningLogs/{docId}/items/{itemId}
```

1. 全明細をCSSに保存する

2. 必ず1つの巨大配列にする

OK 3. サブコレクションに分ける

4. 毎回プロジェクトを作り直す

正解: 3. サブコレクションに分ける

ヒント

親ドキュメントの下に子コレクションを作る考え方です。

解説

サブコレクションにすると明細単位の追加、更新、削除、監視がしやすくなります。ただし読み取り回数やクエリ設計も考える必要があります。

152 FSQ-104

サブコレクション

中級

全範囲

settingsの1件に紐づく明細を多数保存し、明細単位で更新したい。検討しやすい設計は？

```
settings/{docId}/items/{itemId}
```

1. 全明細をCSSに保存する

2. 必ず1つの巨大配列にする

3. 毎回プロジェクトを作り直す

OK 4. サブコレクションに分ける

正解: 4. サブコレクションに分ける

ヒント

親ドキュメントの下に子コレクションを作る考え方です。

解説

サブコレクションにすると明細単位の追加、更新、削除、監視がしやすくなります。ただし読み取り回数やクエリ設計も考える必要があります。

153 FSQ-195

サブコレクション

応用

全範囲

users/{userId}/logs のようなパスで、親に紐づく子データを保存する設計を何と呼ぶ？

users/{userId}/logs

1. 複合インデックス

2. Security Rulesだけ

OK 3. サブコレクション

4. Cloud Storage

正解: 3. サブコレクション

ヒント

親ドキュメントの下にあるコレクションです。

解説

サブコレクションは親ドキュメントに紐づく子データを保存する設計です。明細や回答、メッセージのような繰り返しデータで検討します。

154 FSQ-196

サブコレクション

初級

全範囲

orders/{orderId}/items のようなパスで、親に紐づく子データを保存する設計を何と呼ぶ？

orders/{orderId}/items

1. 複合インデックス

2. Security Rulesだけ

3. Cloud Storage

OK 4. サブコレクション

正解: 4. サブコレクション

ヒント

親ドキュメントの下にあるコレクションです。

解説

サブコレクションは親ドキュメントに紐づく子データを保存する設計です。明細や回答、メッセージのような繰り返しデータで検討します。

155 FSQ-197

サブコレクション

初級

全範囲

stores/{storeId}/menus のようなパスで、親に紐づく子データを保存する設計を何と呼ぶ？

stores/{storeId}/menus

OK 1. サブコレクション

2. 複合インデックス

3. Security Rulesだけ

4. Cloud Storage

正解: 1. サブコレクション

ヒント

親ドキュメントの下にあるコレクションです。

解説

サブコレクションは親ドキュメントに紐づく子データを保存する設計です。明細や回答、メッセージのような繰り返しデータで検討します。

156 FSQ-198

サブコレクション

中級

全範囲

classes/{classId}/students のようなパスで、親に紐づく子データを保存する設計を何と呼ぶ？

classes/{classId}/students

1. 複合インデックス

OK 2. サブコレクション

3. Security Rulesだけ

4. Cloud Storage

正解: 2. サブコレクション

ヒント

親ドキュメントの下にあるコレクションです。

解説

サブコレクションは親ドキュメントに紐づく子データを保存する設計です。明細や回答、メッセージのような繰り返しデータで検討します。

157 FSQ-199

サブコレクション

中級

全範囲

quizzes/{quizId}/answers のようなパスで、親に紐づく子データを保存する設計を何と呼ぶ？

quizzes/{quizId}/answers

1. 複合インデックス

2. Security Rulesだけ

OK 3. サブコレクション

4. Cloud Storage

正解: 3. サブコレクション

ヒント

親ドキュメントの下にあるコレクションです。

解説

サブコレクションは親ドキュメントに紐づく子データを保存する設計です。明細や回答、メッセージのような繰り返しデータで検討します。

158 FSQ-200

サブコレクション

応用

全範囲

tables/{tableId}/orders のようなパスで、親に紐づく子データを保存する設計を何と呼ぶ？

tables/{tableId}/orders

1. 複合インデックス

2. Security Rulesだけ

3. Cloud Storage

OK 4. サブコレクション

正解: 4. サブコレクション

ヒント

親ドキュメントの下にあるコレクションです。

解説

サブコレクションは親ドキュメントに紐づく子データを保存する設計です。明細や回答、メッセージのような繰り返しデータで検討します。

159 FSQ-201

サブコレクション

初級

weak

staff/{staffId}/shifts のようなパスで、親に紐づく子データを保存する設計を何と呼ぶ？

staff/{staffId}/shifts

1. Security Rulesだけ

2. 複合インデックス

3. Cloud Storage

OK 4. サブコレクション

正解: 4. サブコレクション

ヒント

親ドキュメントの下にあるコレクションです。

解説

サブコレクションは親ドキュメントに紐づく子データを保存する設計です。明細や回答、メッセージのような繰り返しデータで検討します。

160 FSQ-202

サブコレクション

初級

全範囲

rooms/{roomId}/messages のようなパスで、親に紐づく子データを保存する設計を何と呼ぶ？

rooms/{roomId}/messages

1. 複合インデックス

OK 2. サブコレクション

3. Security Rulesだけ

4. Cloud Storage

正解: 2. サブコレクション

ヒント

親ドキュメントの下にあるコレクションです。

解説

サブコレクションは親ドキュメントに紐づく子データを保存する設計です。明細や回答、メッセージのような繰り返しデータで検討します。

トランザクション

8問 / 正解番号・正解テキスト・ヒント・解説付き

161 FSQ-105

トランザクション

応用

実装判断

在庫数を複数端末が同時に更新する可能性がある。競合対策として向いているものは？

```
await runTransaction(db, async (transaction) => { ... });
```

OK 1. runTransaction

2. getDocsだけ

3. CSSアニメーション

4. alertだけ

正解: 1. runTransaction

ヒント

読み取りと書き込みを一連の処理として扱います。

解説

transactionは現在値を読み取り、条件を確認してから更新できます。同時更新が起きた場合の再試行にも対応しやすいです。

162 FSQ-106

トランザクション

初級

実装判断

座席数を複数端末が同時に更新する可能性がある。競合対策として向いているものは？

```
await runTransaction(db, async (transaction) => { ... });
```

1. getDocsだけ

OK 2. runTransaction

3. CSSアニメーション

4. alertだけ

正解: 2. runTransaction

ヒント

読み取りと書き込みを一連の処理として扱います。

解説

transactionは現在値を読み取り、条件を確認してから更新できます。同時更新が起きた場合の再試行にも対応しやすいです。

163 FSQ-107

トランザクション

初級

実装判断

注文合計を複数端末が同時に更新する可能性がある。競合対策として向いているものは？

```
await runTransaction(db, async (transaction) => { ... });
```

1. getDocsだけ

2. CSSアニメーション

OK 3. runTransaction

4. alertだけ

正解: 3. runTransaction

ヒント

読み取りと書き込みを一連の処理として扱います。

解説

transactionは現在値を読み取り、条件を確認してから更新できます。同時更新が起きた場合の再試行にも対応しやすいです。

164 FSQ-108

トランザクション

中級

実装判断

回答回数を複数端末が同時に更新する可能性がある。競合対策として向いているものは？

```
await runTransaction(db, async (transaction) => { ... });
```

1. getDocsだけ

2. CSSアニメーション

3. alertだけ

OK 4. runTransaction

正解: 4. runTransaction

ヒント

読み取りと書き込みを一連の処理として扱います。

解説

transactionは現在値を読み取り、条件を確認してから更新できます。同時更新が起きた場合の再試行にも対応しやすいです。

165 FSQ-109

トランザクション

中級

実装判断

正解数を複数端末が同時に更新する可能性がある。競合対策として向いているものは？

```
await runTransaction(db, async (transaction) => { ... });
```

OK 1. runTransaction

2. getDocsだけ

3. CSSアニメーション

4. alertだけ

正解: 1. runTransaction

ヒント

読み取りと書き込みを一連の処理として扱います。

解説

transactionは現在値を読み取り、条件を確認してから更新できます。同時更新が起きた場合の再試行にも対応しやすいです。

166 FSQ-110

トランザクション

応用

実装判断

提供待ち件数を複数端末が同時に更新する可能性がある。競合対策として向いているものは？

```
await runTransaction(db, async (transaction) => { ... });
```

1. getDocsだけ

OK 2. runTransaction

3. CSSアニメーション

4. alertだけ

正解: 2. runTransaction

ヒント

読み取りと書き込みを一連の処理として扱います。

解説

transactionは現在値を読み取り、条件を確認してから更新できます。同時更新が起きた場合の再試行にも対応しやすいです。

167 FSQ-111

トランザクション

初級

実装判断

残数を複数端末が同時に更新する可能性がある。競合対策として向いているものは？

```
await runTransaction(db, async (transaction) => { ... });
```

1. getDocsだけ

2. CSSアニメーション

OK 3. runTransaction

4. alertだけ

正解: 3. runTransaction

ヒント

読み取りと書き込みを一連の処理として扱います。

解説

transactionは現在値を読み取り、条件を確認してから更新できます。同時更新が起きた場合の再試行にも対応しやすいです。

168 FSQ-112

トランザクション

初級

実装判断

ポイントを複数端末が同時に更新する可能性がある。競合対策として向いているものは？

```
await runTransaction(db, async (transaction) => { ... });
```

1. getDocsだけ

2. CSSアニメーション

3. alertだけ

OK 4. runTransaction

正解: 4. runTransaction

ヒント

読み取りと書き込みを一連の処理として扱います。

解説

transactionは現在値を読み取り、条件を確認してから更新できます。同時更新が起きた場合の再試行にも対応しやすいです。

バッチ書き込み

8問 / 正解番号・正解テキスト・ヒント・解説付き

169 FSQ-113

バッチ書き込み

中級

全範囲

合算で複数ドキュメントをまとめて書き込みたい。Firestoreで検討するものは？

```
const batch = writeBatch(db);
```

OK 1. writeBatch

2. where

3. orderBy

4. getDoc

正解: 1. writeBatch

ヒント

複数書き込みをまとめる仕組みです。

解説

writeBatchは複数のset、update、deleteをまとめて実行できます。読み取り結果に応じた条件分岐が必要な場合はtransactionも検討します。

170 FSQ-114

バッチ書き込み

中級

全範囲

テーブル変更で複数ドキュメントをまとめて書き込みたい。Firestoreで検討するものは？

```
const batch = writeBatch(db);
```

1. where

OK 2. writeBatch

3. orderBy

4. getDoc

正解: 2. writeBatch

ヒント

複数書き込みをまとめる仕組みです。

解説

writeBatchは複数のset、update、deleteをまとめて実行できます。読み取り結果に応じた条件分岐が必要な場合はtransactionも検討します。

171 FSQ-115

バッチ書き込み

応用

全範囲

注文取消で複数ドキュメントをまとめて書き込みたい。Firestoreで検討するのは？

```
const batch = writeBatch(db);
```

1. where

2. orderBy

OK 3. writeBatch

4. getDoc

正解: 3. writeBatch

ヒント

複数書き込みをまとめる仕組みです。

解説

writeBatchは複数のset、update、deleteをまとめて実行できます。読み取り結果に応じた条件分岐が必要な場合はtransactionも検討します。

172 FSQ-116

バッチ書き込み

初級

全範囲

提供完了で複数ドキュメントをまとめて書き込みたい。Firestoreで検討するのは？

```
const batch = writeBatch(db);
```

1. where

2. orderBy

3. getDoc

OK 4. writeBatch

正解: 4. writeBatch

ヒント

複数書き込みをまとめる仕組みです。

解説

writeBatchは複数のset、update、deleteをまとめて実行できます。読み取り結果に応じた条件分岐が必要な場合はtransactionも検討します。

173 FSQ-117

バッチ書き込み

初級

全範囲

初期データ投入で複数ドキュメントをまとめて書き込みたい。Firestoreで検討するのは？

```
const batch = writeBatch(db);
```

OK 1. writeBatch

2. where

3. orderBy

4. getDoc

正解: 1. writeBatch

ヒント

複数書き込みをまとめる仕組みです。

解説

writeBatchは複数のset、update、deleteをまとめて実行できます。読み取り結果に応じた条件分岐が必要な場合はtransactionも検討します。

174 FSQ-118

バッチ書き込み

中級

全範囲

一括状態更新で複数ドキュメントをまとめて書き込みたい。Firestoreで検討するのは？

```
const batch = writeBatch(db);
```

1. where

OK 2. writeBatch

3. orderBy

4. getDoc

正解: 2. writeBatch

ヒント

複数書き込みをまとめる仕組みです。

解説

writeBatchは複数のset、update、deleteをまとめて実行できます。読み取り結果に応じた条件分岐が必要な場合はtransactionも検討します。

175 FSQ-119

バッチ書き込み

中級

全範囲

複数ログ保存で複数ドキュメントをまとめて書き込みたい。Firestoreで検討するのは？

```
const batch = writeBatch(db);
```

1. where

2. orderBy

OK 3. writeBatch

4. getDoc

正解: 3. writeBatch

ヒント

複数書き込みをまとめる仕組みです。

解説

writeBatchは複数のset、update、deleteをまとめて実行できます。読み取り結果に応じた条件分岐が必要な場合はtransactionも検討します。

176 FSQ-120

バッチ書き込み

応用

daily

メニュー一括更新で複数ドキュメントをまとめて書き込みたい。Firestoreで検討するのは？

```
const batch = writeBatch(db);
```

1. getDoc

2. where

OK 3. writeBatch

4. orderBy

正解: 3. writeBatch

ヒント

複数書き込みをまとめる仕組みです。

解説

writeBatchは複数のset、update、deleteをまとめて実行できます。読み取り結果に応じた条件分岐が必要な場合はtransactionも検討します。

セキュリティルール

16問 / 正解番号・正解テキスト・ヒント・解説付き

177 FSQ-121

セキュリティルール

初級

実装判断

注文作成をFirestoreで許可するか制御したい。本番で設定すべきものは？

```
match /databases/{database}/documents { ... }
```

OK 1. Security Rules

2. CSSのdisplay:none

3. HTMLコメント

4. favicon設定

正解: 1. Security Rules

ヒント

画面側の非表示だけでは保護になりません。

解説

Firestoreの読み書き権限はSecurity Rulesで制御します。クライアントから直接アクセスする場合は特に重要です。

178 FSQ-122

セキュリティルール

初級

実装判断

注文更新をFirestoreで許可するか制御したい。本番で設定すべきものは？

```
match /databases/{database}/documents { ... }
```

1. CSSのdisplay:none

OK 2. Security Rules

3. HTMLコメント

4. favicon設定

正解: 2. Security Rules

ヒント

画面側の非表示だけでは保護になりません。

解説

Firestoreの読み書き権限はSecurity Rulesで制御します。クライアントから直接アクセスする場合は特に重要です。

179 FSQ-123

セキュリティルール

中級

実装判断

注文削除をFirestoreで許可するか制御したい。本番で設定すべきものは？

```
match /databases/{database}/documents { ... }
```

1. CSSのdisplay:none

2. HTMLコメント

OK 3. Security Rules

4. favicon設定

正解: 3. Security Rules

ヒント

画面側の非表示だけでは保護になりません。

解説

Firestoreの読み書き権限はSecurity Rulesで制御します。クライアントから直接アクセスする場合は特に重要です。

180 FSQ-124

セキュリティルール

中級

実装判断

学習ログ閲覧をFirestoreで許可するか制御したい。本番で設定すべきものは？

```
match /databases/{database}/documents { ... }
```

1. CSSのdisplay:none

2. HTMLコメント

3. favicon設定

OK 4. Security Rules

正解: 4. Security Rules

ヒント

画面側の非表示だけでは保護になりません。

解説

Firestoreの読み書き権限はSecurity Rulesで制御します。クライアントから直接アクセスする場合は特に重要です。

181 FSQ-125

セキュリティルール

応用

実装判断

問題マスタ編集をFirestoreで許可するか制御したい。本番で設定すべきものは？

```
match /databases/{database}/documents { ... }
```

OK 1. Security Rules

2. CSSのdisplay:none

3. HTMLコメント

4. favicon設定

正解: 1. Security Rules

ヒント

画面側の非表示だけでは保護になりません。

解説

Firestoreの読み書き権限はSecurity Rulesで制御します。クライアントから直接アクセスする場合は特に重要です。

182 FSQ-126

セキュリティルール

初級

実装判断

ユーザー進捗閲覧をFirestoreで許可するか制御したい。本番で設定すべきものは？

```
match /databases/{database}/documents { ... }
```

1. CSSのdisplay:none

OK 2. Security Rules

3. HTMLコメント

4. favicon設定

正解: 2. Security Rules

ヒント

画面側の非表示だけでは保護になりません。

解説

Firestoreの読み書き権限はSecurity Rulesで制御します。クライアントから直接アクセスする場合は特に重要です。

183 FSQ-127

セキュリティルール

初級

実装判断

メニュー更新をFirestoreで許可するか制御したい。本番で設定すべきものは？

```
match /databases/{database}/documents { ... }
```

1. CSSのdisplay:none

2. HTMLコメント

OK 3. Security Rules

4. favicon設定

正解: 3. Security Rules

ヒント

画面側の非表示だけでは保護になりません。

解説

Firestoreの読み書き権限はSecurity Rulesで制御します。クライアントから直接アクセスする場合は特に重要です。

184 FSQ-128

セキュリティルール

中級

実装判断

スタッフ情報閲覧をFirestoreで許可するか制御したい。本番で設定すべきものは？

```
match /databases/{database}/documents { ... }
```

1. CSSのdisplay:none

2. HTMLコメント

3. favicon設定

OK 4. Security Rules

正解: 4. Security Rules

ヒント

画面側の非表示だけでは保護になりません。

解説

Firestoreの読み書き権限はSecurity Rulesで制御します。クライアントから直接アクセスする場合は特に重要です。

185 FSQ-211

セキュリティルール

初級

実装判断

Security Rulesの条件「request.auth != null」は何を目的に使う？

```
request.auth != null
```

1. CSSの色を変えるため

2. Excelの列幅を決めるため

OK 3. Firestoreの読み書き可否を判定するため

4. ブラウザの戻るボタンを止めるため

正解: 3. Firestoreの読み書き可否を判定するため

ヒント

Rulesはデータアクセスの入口で評価されます。

解説

Security RulesはFirestoreへの読み書きを許可するか判定します。認証状態、ユーザーID、保存データの形などを条件にできます。

186 FSQ-212

セキュリティルール

初級

実装判断

Security Rulesの条件「request.auth.uid == userId」は何を目的に使う？

```
request.auth.uid == userId
```

1. CSSの色を変えるため

2. Excelの列幅を決めるため

3. ブラウザの戻るボタンを止めるため

OK 4. Firestoreの読み書き可否を判定するため

正解: 4. Firestoreの読み書き可否を判定するため

ヒント

Rulesはデータアクセスの入口で評価されます。

解説

Security RulesはFirestoreへの読み書きを許可するか判定します。認証状態、ユーザーID、保存データの形などを条件にできます。

187 FSQ-213

セキュリティルール

中級

実装判断

Security Rulesの条件「resource.data.ownerId == request.auth.uid」は何を目的に使う？

```
resource.data.ownerId == request.auth.uid
```

OK 1. Firestoreの読み書き可否を判定するため

2. CSSの色を変えるため

3. Excelの列幅を決めるため

4. ブラウザの戻るボタンを止めるため

正解: 1. Firestoreの読み書き可否を判定するため

ヒント

Rulesはデータアクセスの入口で評価されます。

解説

Security RulesはFirestoreへの読み書きを許可するか判定します。認証状態、ユーザーID、保存データの形などを条件にできます。

188 FSQ-214

セキュリティルール

中級

実装判断

Security Rulesの条件「request.resource.data.keys().hasOnly([...])」は何を目的に使う？

```
request.resource.data.keys().hasOnly([...])
```

1. CSSの色を変えるため

OK 2. Firestoreの読み書き可否を判定するため

3. Excelの列幅を決めるため

4. ブラウザの戻るボタンを止めるため

正解: 2. Firestoreの読み書き可否を判定するため

ヒント

Rulesはデータアクセスの入口で評価されます。

解説

Security RulesはFirestoreへの読み書きを許可するか判定します。認証状態、ユーザーID、保存データの形などを条件にできます。

189 FSQ-215

セキュリティルール

応用

実装判断

Security Rulesの条件「request.time < timestamp.date(2030, 1, 1)」は何を目的に使う？

```
request.time < timestamp.date(2030, 1, 1)
```

1. CSSの色を変えるため

2. Excelの列幅を決めるため

OK 3. Firestoreの読み書き可否を判定するため

4. ブラウザの戻るボタンを止めるため

正解: 3. Firestoreの読み書き可否を判定するため

ヒント

Rulesはデータアクセスの入口で評価されます。

解説

Security RulesはFirestoreへの読み書きを許可するか判定します。認証状態、ユーザーID、保存データの形などを条件にできます。

190 FSQ-216

セキュリティルール

初級

実装判断

Security Rulesの条件「get(/databases/\$(database)/documents/users/\$(request.auth.uid))」は何を目的に使う？

```
get(/databases/$(database)/documents/users/$(request.auth.uid))
```

1. CSSの色を変えるため

2. Excelの列幅を決めるため

3. ブラウザの戻るボタンを止めるため

OK 4. Firestoreの読み書き可否を判定するため

正解: 4. Firestoreの読み書き可否を判定するため

ヒント

Rulesはデータアクセスの入口で評価されます。

解説

Security RulesはFirestoreへの読み書きを許可するか判定します。認証状態、ユーザーID、保存データの形などを条件にできます。

191 FSQ-217

セキュリティルール

初級

実装判断

Security Rulesの条件「allow read」は何を目的に使う？

allow read

OK 1. Firestoreの読み書き可否を判定するため

2. CSSの色を変えるため

3. Excelの列幅を決めるため

4. ブラウザの戻るボタンを止めるため

正解: 1. Firestoreの読み書き可否を判定するため

ヒント

Rulesはデータアクセスの入口で評価されます。

解説

Security RulesはFirestoreへの読み書きを許可するか判定します。認証状態、ユーザーID、保存データの形などを条件にできます。

192 FSQ-218

セキュリティルール

中級

実装判断

Security Rulesの条件「allow write」は何を目的に使う？

allow write

1. CSSの色を変えるため

OK 2. Firestoreの読み書き可否を判定するため

3. Excelの列幅を決めるため

4. ブラウザの戻るボタンを止めるため

正解: 2. Firestoreの読み書き可否を判定するため

ヒント

Rulesはデータアクセスの入口で評価されます。

解説

Security RulesはFirestoreへの読み書きを許可するか判定します。認証状態、ユーザーID、保存データの形などを条件にできます。

パフォーマンス

16問 / 正解番号・正解テキスト・ヒント・解説付き

193 FSQ-129

パフォーマンス

中級

全範囲

大量注文一覧をFirestoreから表示するとき、読み取り量を抑える基本方針は？

```
query(ref, orderBy('createdAt', 'desc'), limit(20));
```

OK 1. whereやlimitで必要な範囲だけ取得する

2. 毎回全ドキュメントを読む

3. 画像として保存する

4. CSSで非表示にする

正解: 1. whereやlimitで必要な範囲だけ取得する

ヒント

Firestoreは読み取ったドキュメント数が重要です。

解説

必要な条件で絞り込み、limitで件数を抑えると、表示速度と読み取りコストを管理しやすくなります。

194 FSQ-130

パフォーマンス

応用

全範囲

最新履歴をFirestoreから表示するとき、読み取り量を抑える基本方針は？

```
query(ref, orderBy('createdAt', 'desc'), limit(20));
```

1. 毎回全ドキュメントを読む

OK 2. whereやlimitで必要な範囲だけ取得する

3. 画像として保存する

4. CSSで非表示にする

正解: 2. whereやlimitで必要な範囲だけ取得する

ヒント

Firestoreは読み取ったドキュメント数が重要です。

解説

必要な条件で絞り込み、limitで件数を抑えると、表示速度と読み取りコストを管理しやすくなります。

195 FSQ-131

パフォーマンス

初級

全範囲

カテゴリ別問題をFirestoreから表示するとき、読み取り量を抑える基本方針は？

```
query(ref, orderBy('createdAt', 'desc'), limit(20));
```

1. 毎回全ドキュメントを読む

2. 画像として保存する

OK 3. whereやlimitで必要な範囲だけ取得する

4. CSSで非表示にする

正解: 3. whereやlimitで必要な範囲だけ取得する

ヒント

Firestoreは読み取ったドキュメント数が重要です。

解説

必要な条件で絞り込み、limitで件数を抑えると、表示速度と読み取りコストを管理しやすくなります。

196 FSQ-132

パフォーマンス

初級

全範囲

ユーザー別ログをFirestoreから表示するとき、読み取り量を抑える基本方針は？

```
query(ref, orderBy('createdAt', 'desc'), limit(20));
```

1. 毎回全ドキュメントを読む

2. 画像として保存する

3. CSSで非表示にする

OK 4. whereやlimitで必要な範囲だけ取得する

正解: 4. whereやlimitで必要な範囲だけ取得する

ヒント

Firestoreは読み取ったドキュメント数が重要です。

解説

必要な条件で絞り込み、limitで件数を抑えると、表示速度と読み取りコストを管理しやすくなります。

197 FSQ-133

パフォーマンス

中級

全範囲

提供済み一覧をFirestoreから表示するとき、読み取り量を抑える基本方針は？

```
query(ref, orderBy('createdAt', 'desc'), limit(20));
```

OK 1. whereやlimitで必要な範囲だけ取得する

2. 毎回全ドキュメントを読む

3. 画像として保存する

4. CSSで非表示にする

正解: 1. whereやlimitで必要な範囲だけ取得する

ヒント

Firestoreは読み取ったドキュメント数が重要です。

解説

必要な条件で絞り込み、limitで件数を抑えると、表示速度と読み取りコストを管理しやすくなります。

198 FSQ-134

パフォーマンス

中級

全範囲

KDS待ち一覧をFirestoreから表示するとき、読み取り量を抑える基本方針は？

```
query(ref, orderBy('createdAt', 'desc'), limit(20));
```

1. 毎回全ドキュメントを読む

OK 2. whereやlimitで必要な範囲だけ取得する

3. 画像として保存する

4. CSSで非表示にする

正解: 2. whereやlimitで必要な範囲だけ取得する

ヒント

Firestoreは読み取ったドキュメント数が重要です。

解説

必要な条件で絞り込み、limitで件数を抑えると、表示速度と読み取りコストを管理しやすくなります。

199 FSQ-135

パフォーマンス

応用

全範囲

ランキングをFirestoreから表示するとき、読み取り量を抑える基本方針は？

```
query(ref, orderBy('createdAt', 'desc'), limit(20));
```

1. 毎回全ドキュメントを読む

2. 画像として保存する

OK 3. whereやlimitで必要な範囲だけ取得する

4. CSSで非表示にする

正解: 3. whereやlimitで必要な範囲だけ取得する

ヒント

Firestoreは読み取ったドキュメント数が重要です。

解説

必要な条件で絞り込み、limitで件数を抑えると、表示速度と読み取りコストを管理しやすくなります。

200 FSQ-136

パフォーマンス

初級

全範囲

検索結果をFirestoreから表示するとき、読み取り量を抑える基本方針は？

```
query(ref, orderBy('createdAt', 'desc'), limit(20));
```

1. 毎回全ドキュメントを読む

2. 画像として保存する

3. CSSで非表示にする

OK 4. whereやlimitで必要な範囲だけ取得する

正解: 4. whereやlimitで必要な範囲だけ取得する

ヒント

Firestoreは読み取ったドキュメント数が重要です。

解説

必要な条件で絞り込み、limitで件数を抑えると、表示速度と読み取りコストを管理しやすくなります。

201 FSQ-171

パフォーマンス

初級

全範囲

Firestoreから最新5件だけ取得したい。読み取り量を抑えるために使うものは？

```
query(collection(db, 'orders'), orderBy('createdAt', 'desc'), limit(5));
```

1. deleteDoc(5)

2. serverTimestamp(5)

OK 3. limit(5)

4. collection(5)

正解: 3. limit(5)

ヒント

取得件数を制限します。

解説

limitを使うと取得件数を制限できます。大量のドキュメントを毎回読むより、読み取り量と表示速度を管理しやすくなります。

202 FSQ-172

パフォーマンス

初級

全範囲

Firestoreから最新10件だけ取得したい。読み取り量を抑えるために使うものは？

```
query(collection(db, 'orders'), orderBy('createdAt', 'desc'), limit(10));
```

1. deleteDoc(10)

2. serverTimestamp(10)

3. collection(10)

OK 4. limit(10)

正解: 4. limit(10)

ヒント

取得件数を制限します。

解説

limitを使うと取得件数を制限できます。大量のドキュメントを毎回読むより、読み取り量と表示速度を管理しやすくなります。

203 FSQ-173

パフォーマンス

中級

全範囲

Firestoreから最新20件だけ取得したい。読み取り量を抑えるために使うものは？

```
query(collection(db, 'orders'), orderBy('createdAt', 'desc'), limit(20));
```

OK 1. limit(20)

2. deleteDoc(20)

3. serverTimestamp(20)

4. collection(20)

正解: 1. limit(20)

ヒント

取得件数を制限します。

解説

limitを使うと取得件数を制限できます。大量のドキュメントを毎回読むより、読み取り量と表示速度を管理しやすくなります。

204 FSQ-174

パフォーマンス

中級

全範囲

Firestoreから最新30件だけ取得したい。読み取り量を抑えるために使うものは？

```
query(collection(db, 'orders'), orderBy('createdAt', 'desc'), limit(30));
```

1. deleteDoc(30)

OK 2. limit(30)

3. serverTimestamp(30)

4. collection(30)

正解: 2. limit(30)

ヒント

取得件数を制限します。

解説

limitを使うと取得件数を制限できます。大量のドキュメントを毎回読むより、読み取り量と表示速度を管理しやすくなります。

205 FSQ-175

パフォーマンス

応用

全範囲

Firestoreから最新50件だけ取得したい。読み取り量を抑えるために使うものは？

```
query(collection(db, 'orders'), orderBy('createdAt', 'desc'), limit(50));
```

1. deleteDoc(50)

2. serverTimestamp(50)

OK 3. limit(50)

4. collection(50)

正解: 3. limit(50)

ヒント

取得件数を制限します。

解説

limitを使うと取得件数を制限できます。大量のドキュメントを毎回読むより、読み取り量と表示速度を管理しやすくなります。

206 FSQ-176

パフォーマンス

初級

全範囲

Firestoreから最新100件だけ取得したい。読み取り量を抑えるために使うものは？

```
query(collection(db, 'orders'), orderBy('createdAt', 'desc'), limit(100));
```

1. deleteDoc(100)

2. serverTimestamp(100)

3. collection(100)

OK 4. limit(100)

正解: 4. limit(100)

ヒント

取得件数を制限します。

解説

limitを使うと取得件数を制限できます。大量のドキュメントを毎回読むより、読み取り量と表示速度を管理しやすくなります。

207 FSQ-177

パフォーマンス

初級

daily

Firestoreから最新200件だけ取得したい。読み取り量を抑えるために使うものは？

```
query(collection(db, 'orders'), orderBy('createdAt', 'desc'), limit(200));
```

OK 1. limit(200)

2. deleteDoc(200)

3. serverTimestamp(200)

4. collection(200)

正解: 1. limit(200)

ヒント

取得件数を制限します。

解説

limitを使うと取得件数を制限できます。大量のドキュメントを毎回読むより、読み取り量と表示速度を管理しやすくなります。

208 FSQ-178

パフォーマンス

中級

全範囲

Firestoreから最新500件だけ取得したい。読み取り量を抑えるために使うものは？

```
query(collection(db, 'orders'), orderBy('createdAt', 'desc'), limit(500));
```

OK 2. limit(500)

1. deleteDoc(500)

3. serverTimestamp(500)

4. collection(500)

正解: 2. limit(500)

ヒント

取得件数を制限します。

解説

limitを使うと取得件数を制限できます。大量のドキュメントを毎回読むより、読み取り量と表示速度を管理しやすくなります。

エラーハンドリング

16問 / 正解番号・正解テキスト・ヒント・解説付き

209 FSQ-137

エラーハンドリング

初級

全範囲

Firestore操作で「権限不足」が起きた。実装として重要な対応は？

```
try { await updateDoc(ref, data); } catch (error) { ... }
```

OK 1. catchで失敗を扱い、原因に応じた表示や再試行を行う

2. 常に成功扱いにする

3. 何も表示しない

4. 無関係なドキュメントを削除する

正解: 1. catchで失敗を扱い、原因に応じた表示や再試行を行う

ヒント

通信や権限の失敗は起こる前提で扱います。

解説

Firestore操作はネットワーク、権限、インデックス、データ型などで失敗します。catchで扱い、ユーザーに分かる形で伝えることが大切です。

210 FSQ-138

エラーハンドリング

中級

全範囲

Firestore操作で「インデックス不足」が起きた。実装として重要な対応は？

```
try { await updateDoc(ref, data); } catch (error) { ... }
```

1. 常に成功扱いにする

OK 2. catchで失敗を扱い、原因に応じた表示や再試行を行う

3. 何も表示しない

4. 無関係なドキュメントを削除する

正解: 2. catchで失敗を扱い、原因に応じた表示や再試行を行う

ヒント

通信や権限の失敗は起こる前提で扱います。

解説

Firestore操作はネットワーク、権限、インデックス、データ型などで失敗します。catchで扱い、ユーザーに分かる形で伝えることが大切です。

211 FSQ-139

エラーハンドリング

中級

全範囲

Firestore操作で「ネットワーク切断」が起きた。実装として重要な対応は？

```
try { await updateDoc(ref, data); } catch (error) { ... }
```

1. 常に成功扱いにする

2. 何も表示しない

OK 3. catchで失敗を扱い、原因に応じた表示や再試行を行う

4. 無関係なドキュメントを削除する

正解: 3. catchで失敗を扱い、原因に応じた表示や再試行を行う

ヒント

通信や権限の失敗は起こる前提で扱います。

解説

Firestore操作はネットワーク、権限、インデックス、データ型などで失敗します。catchで扱い、ユーザーに分かる形で伝えることが大切です。

212 FSQ-140

エラーハンドリング

応用

全範囲

Firestore操作で「ドキュメントなし」が起きた。実装として重要な対応は？

```
try { await updateDoc(ref, data); } catch (error) { ... }
```

1. 常に成功扱いにする

2. 何も表示しない

3. 無関係なドキュメントを削除する

OK 4. catchで失敗を扱い、原因に応じた表示や再試行を行う

正解: 4. catchで失敗を扱い、原因に応じた表示や再試行を行う

ヒント

通信や権限の失敗は起こる前提で扱います。

解説

Firestore操作はネットワーク、権限、インデックス、データ型などで失敗します。catchで扱い、ユーザーに分かる形で伝えることが大切です。

213 FSQ-141

エラーハンドリング

初級

全範囲

Firestore操作で「型違い」が起きた。実装として重要な対応は？

```
try { await updateDoc(ref, data); } catch (error) { ... }
```

OK 1. catchで失敗を扱い、原因に応じた表示や再試行を行う

2. 常に成功扱いにする

3. 何も表示しない

4. 無関係なドキュメントを削除する

正解: 1. catchで失敗を扱い、原因に応じた表示や再試行を行う

ヒント

通信や権限の失敗は起こる前提で扱います。

解説

Firestore操作はネットワーク、権限、インデックス、データ型などで失敗します。catchで扱い、ユーザーに分かる形で伝えることが大切です。

214 FSQ-142

エラーハンドリング

初級

全範囲

Firestore操作で「不正なパス」が起きた。実装として重要な対応は？

```
try { await updateDoc(ref, data); } catch (error) { ... }
```

1. 常に成功扱いにする

OK 2. catchで失敗を扱い、原因に応じた表示や再試行を行う

3. 何も表示しない

4. 無関係なドキュメントを削除する

正解: 2. catchで失敗を扱い、原因に応じた表示や再試行を行う

ヒント

通信や権限の失敗は起こる前提で扱います。

解説

Firestore操作はネットワーク、権限、インデックス、データ型などで失敗します。catchで扱い、ユーザーに分かる形で伝えることが大切です。

215 FSQ-143

エラーハンドリング

中級

全範囲

Firestore操作で「初期化漏れ」が起きた。実装として重要な対応は？

```
try { await updateDoc(ref, data); } catch (error) { ... }
```

1. 常に成功扱いにする

2. 何も表示しない

OK 3. catchで失敗を扱い、原因に応じた表示や再試行を行う

4. 無関係なドキュメントを削除する

正解: 3. catchで失敗を扱い、原因に応じた表示や再試行を行う

ヒント

通信や権限の失敗は起こる前提で扱います。

解説

Firestore操作はネットワーク、権限、インデックス、データ型などで失敗します。catchで扱い、ユーザーに分かる形で伝えることが大切です。

216 FSQ-144

エラーハンドリング

中級

全範囲

Firestore操作で「同時更新失敗」が起きた。実装として重要な対応は？

```
try { await updateDoc(ref, data); } catch (error) { ... }
```

1. 常に成功扱いにする

2. 何も表示しない

3. 無関係なドキュメントを削除する

OK 4. catchで失敗を扱い、原因に応じた表示や再試行を行う

正解: 4. catchで失敗を扱い、原因に応じた表示や再試行を行う

ヒント

通信や権限の失敗は起こる前提で扱います。

解説

Firestore操作はネットワーク、権限、インデックス、データ型などで失敗します。catchで扱い、ユーザーに分かる形で伝えることが大切です。

217 FSQ-219

エラーハンドリング

中級

全範囲

Firestoreエラーコード「permission-denied」が返った。実装で最も適切な対応は？

1. 常に成功として扱う

2. 関係ないデータを削除する

OK 3. エラー内容に応じて表示、再試行、設定確認を分ける

4. 選択肢を全部正解にする

正解: 3. エラー内容に応じて表示、再試行、設定確認を分ける

ヒント

エラーコードごとに原因が違います。

解説

Firestoreのエラーは権限、存在しないデータ、インデックス不足、通信不安定など原因が分かります。catchで受けて適切に扱います。

218 FSQ-220

エラーハンドリング

応用

全範囲

Firestoreエラーコード「not-found」が返った。実装で最も適切な対応は？

1. 常に成功として扱う

2. 関係ないデータを削除する

3. 選択肢を全部正解にする

OK 4. エラー内容に応じて表示、再試行、設定確認を分ける

正解: 4. エラー内容に応じて表示、再試行、設定確認を分ける

ヒント

エラーコードごとに原因が違います。

解説

Firestoreのエラーは権限、存在しないデータ、インデックス不足、通信不安定など原因が分かります。catchで受けて適切に扱います。

219 FSQ-221

エラーハンドリング

初級

全範囲

Firestoreエラーコード「already-exists」が返った。実装で最も適切な対応は？

OK 1. エラー内容に応じて表示、再試行、設定確認を分ける

2. 常に成功として扱う

3. 関係ないデータを削除する

4. 選択肢を全部正解にする

正解: 1. エラー内容に応じて表示、再試行、設定確認を分ける

ヒント

エラーコードごとに原因が違います。

解説

Firestoreのエラーは権限、存在しないデータ、インデックス不足、通信不安定など原因が分かります。catchで受けて適切に扱います。

220 FSQ-222

エラーハンドリング

初級

全範囲

Firestoreエラーコード「resource-exhausted」が返った。実装で最も適切な対応は？

1. 常に成功として扱う

OK 2. エラー内容に応じて表示、再試行、設定確認を分ける

3. 関係ないデータを削除する

4. 選択肢を全部正解にする

正解: 2. エラー内容に応じて表示、再試行、設定確認を分ける

ヒント

エラーコードごとに原因が違います。

解説

Firestoreのエラーは権限、存在しないデータ、インデックス不足、通信不安定など原因が分かります。catchで受けて適切に扱います。

221 FSQ-223

エラーハンドリング

中級

全範囲

Firestoreエラーコード「failed-precondition」が返った。実装で最も適切な対応は？

1. 常に成功として扱う

2. 関係ないデータを削除する

OK 3. エラー内容に応じて表示、再試行、設定確認を分ける

4. 選択肢を全部正解にする

正解: 3. エラー内容に応じて表示、再試行、設定確認を分ける

ヒント

エラーコードごとに原因が違います。

解説

Firestoreのエラーは権限、存在しないデータ、インデックス不足、通信不安定など原因が分かれます。catchで受けて適切に扱います。

222 FSQ-224

エラーハンドリング

中級

全範囲

Firestoreエラーコード「unavailable」が返った。実装で最も適切な対応は？

1. 常に成功として扱う

2. 関係ないデータを削除する

3. 選択肢を全部正解にする

OK 4. エラー内容に応じて表示、再試行、設定確認を分ける

正解: 4. エラー内容に応じて表示、再試行、設定確認を分ける

ヒント

エラーコードごとに原因が違います。

解説

Firestoreのエラーは権限、存在しないデータ、インデックス不足、通信不安定など原因が分かれます。catchで受けて適切に扱います。

223 FSQ-225

エラーハンドリング

応用

全範囲

Firestoreエラーコード「deadline-exceeded」が返った。実装で最も適切な対応は？

OK 1. エラー内容に応じて表示、再試行、設定確認を分ける

2. 常に成功として扱う

3. 関係ないデータを削除する

4. 選択肢を全部正解にする

正解: 1. エラー内容に応じて表示、再試行、設定確認を分ける

ヒント

エラーコードごとに原因が違います。

解説

Firestoreのエラーは権限、存在しないデータ、インデックス不足、通信不安定など原因が分かります。catchで受けて適切に扱います。

224 FSQ-226

エラーハンドリング

初級

全範囲

Firestoreエラーコード「cancelled」が返った。実装で最も適切な対応は？

1. 常に成功として扱う

OK 2. エラー内容に応じて表示、再試行、設定確認を分ける

3. 関係ないデータを削除する

4. 選択肢を全部正解にする

正解: 2. エラー内容に応じて表示、再試行、設定確認を分ける

ヒント

エラーコードごとに原因が違います。

解説

Firestoreのエラーは権限、存在しないデータ、インデックス不足、通信不安定など原因が分かります。catchで受けて適切に扱います。

オフライン対応

8問 / 正解番号・正解テキスト・ヒント・解説付き

225 FSQ-145

オフライン対応

応用

全範囲

注文作成を通信が不安定な環境で扱うとき、Firestore利用時に意識したいことは？

OK 1. 失敗時の表示、再試行、二重送信防止を設計する

2. 背景色だけ変える

3. 成功したことにする

4. 全データを毎回削除する

正解: 1. 失敗時の表示、再試行、二重送信防止を設計する

ヒント

オンライン前提だけで作ると、現場で困ることがあります。

解説

Firestoreにはオフラインキャッシュ機能もありますが、業務処理では成功確認、再試行、二重送信防止を画面側でも考える必要があります。

226 FSQ-146

オフライン対応

初級

全範囲

注文更新を通信が不安定な環境で扱うとき、Firestore利用時に意識したいことは？

1. 背景色だけ変える

OK 2. 失敗時の表示、再試行、二重送信防止を設計する

3. 成功したことにする

4. 全データを毎回削除する

正解: 2. 失敗時の表示、再試行、二重送信防止を設計する

ヒント

オンライン前提だけで作ると、現場で困ることがあります。

解説

Firestoreにはオフラインキャッシュ機能もありますが、業務処理では成功確認、再試行、二重送信防止を画面側でも考える必要があります。

227 FSQ-147

オフライン対応

初級

全範囲

ログ保存を通信が不安定な環境で扱うとき、Firestore利用時に意識したいことは？

1. 背景色だけ変える

2. 成功したことにする

OK 3. 失敗時の表示、再試行、二重送信防止を設計する

4. 全データを毎回削除する

正解: 3. 失敗時の表示、再試行、二重送信防止を設計する

ヒント

オンライン前提だけで作ると、現場で困ることがあります。

解説

Firestoreにはオフラインキャッシュ機能もありますが、業務処理では成功確認、再試行、二重送信防止を画面側でも考える必要があります。

228 FSQ-148

オフライン対応

中級

全範囲

問題取得を通信が不安定な環境で扱うとき、Firestore利用時に意識したいことは？

1. 背景色だけ変える

2. 成功したことにする

3. 全データを毎回削除する

OK 4. 失敗時の表示、再試行、二重送信防止を設計する

正解: 4. 失敗時の表示、再試行、二重送信防止を設計する

ヒント

オンライン前提だけで作ると、現場で困ることがあります。

解説

Firestoreにはオフラインキャッシュ機能もありますが、業務処理では成功確認、再試行、二重送信防止を画面側でも考える必要があります。

229 FSQ-149

オフライン対応

中級

全範囲

KDS表示を通信が不安定な環境で扱うとき、Firestore利用時に意識したいことは？

OK 1. 失敗時の表示、再試行、二重送信防止を設計する

2. 背景色だけ変える

3. 成功したことにする

4. 全データを毎回削除する

正解: 1. 失敗時の表示、再試行、二重送信防止を設計する

ヒント

オンライン前提だけで作ると、現場で困ることがあります。

解説

Firestoreにはオフラインキャッシュ機能もありますが、業務処理では成功確認、再試行、二重送信防止を画面側でも考える必要があります。

230 FSQ-150

オフライン対応

応用

全範囲

履歴確認を通信が不安定な環境で扱うとき、Firestore利用時に意識したいことは？

1. 背景色だけ変える

OK 2. 失敗時の表示、再試行、二重送信防止を設計する

3. 成功したことにする

4. 全データを毎回削除する

正解: 2. 失敗時の表示、再試行、二重送信防止を設計する

ヒント

オンライン前提だけで作ると、現場で困ることがあります。

解説

Firestoreにはオフラインキャッシュ機能もありますが、業務処理では成功確認、再試行、二重送信防止を画面側でも考える必要があります。

231 FSQ-151

オフライン対応

初級

全範囲

メニュー読込を通信が不安定な環境で扱うとき、Firestore利用時に意識したいことは？

1. 背景色だけ変える

2. 成功したことにする

OK 3. 失敗時の表示、再試行、二重送信防止を設計する

4. 全データを毎回削除する

正解: 3. 失敗時の表示、再試行、二重送信防止を設計する

ヒント

オンライン前提だけで作ると、現場で困ることがあります。

解説

Firestoreにはオフラインキャッシュ機能もありますが、業務処理では成功確認、再試行、二重送信防止を画面側でも考える必要があります。

232 FSQ-152

オフライン対応

初級

全範囲

進捗保存を通信が不安定な環境で扱うとき、Firestore利用時に意識したいことは？

1. 背景色だけ変える

2. 成功したことにする

3. 全データを毎回削除する

OK 4. 失敗時の表示、再試行、二重送信防止を設計する

正解: 4. 失敗時の表示、再試行、二重送信防止を設計する

ヒント

オンライン前提だけで作ると、現場で困ることがあります。

解説

Firestoreにはオフラインキャッシュ機能もありますが、業務処理では成功確認、再試行、二重送信防止を画面側でも考える必要があります。

削除

8問 / 正解番号・正解テキスト・ヒント・解説付き

233 FSQ-153

削除

中級

全範囲

ordersコレクションの特定ドキュメントを削除したい。使うAPIは？

```
await deleteDoc(doc(db, 'orders', docId));
```

OK 1. deleteDoc

2. addDoc

3. getDocs

4. serverTimestamp

正解: 1. deleteDoc

ヒント

削除対象はDocumentReferenceで指定します。

解説

deleteDocは指定したドキュメントを削除します。履歴を残したい業務データでは、物理削除ではなくstatus変更も検討します。

234 FSQ-154

削除

中級

全範囲

menusコレクションの特定ドキュメントを削除したい。使うAPIは？

```
await deleteDoc(doc(db, 'menus', docId));
```

1. addDoc

OK 2. deleteDoc

3. getDocs

4. serverTimestamp

正解: 2. deleteDoc

ヒント

削除対象はDocumentReferenceで指定します。

解説

deleteDocは指定したドキュメントを削除します。履歴を残したい業務データでは、物理削除ではなくstatus変更も検討します。

235 FSQ-155

削除

応用

全範囲

usersコレクションの特定ドキュメントを削除したい。使うAPIは？

```
await deleteDoc(doc(db, 'users', docId));
```

1. addDoc

2. getDocs

OK 3. deleteDoc

4. serverTimestamp

正解: 3. deleteDoc

ヒント

削除対象はDocumentReferenceで指定します。

解説

deleteDocは指定したドキュメントを削除します。履歴を残したい業務データでは、物理削除ではなくstatus変更も検討します。

236 FSQ-156

削除

初級

全範囲

CookListコレクションの特定ドキュメントを削除したい。使うAPIは？

```
await deleteDoc(doc(db, 'CookList', docId));
```

1. addDoc

2. getDocs

3. serverTimestamp

OK 4. deleteDoc

正解: 4. deleteDoc

ヒント

削除対象はDocumentReferenceで指定します。

解説

deleteDocは指定したドキュメントを削除します。履歴を残したい業務データでは、物理削除ではなくstatus変更も検討します。

237 FSQ-157

削除

初級

全範囲

StaffListコレクションの特定ドキュメントを削除したい。使うAPIは？

```
await deleteDoc(doc(db, 'StaffList', docId));
```

OK 1. deleteDoc

2. addDoc

3. getDocs

4. serverTimestamp

正解: 1. deleteDoc

ヒント

削除対象はDocumentReferenceで指定します。

解説

deleteDocは指定したドキュメントを削除します。履歴を残したい業務データでは、物理削除ではなくstatus変更も検討します。

238 FSQ-158

削除

中級

全範囲

quizQuestionsコレクションの特定ドキュメントを削除したい。使うAPIは？

```
await deleteDoc(doc(db, 'quizQuestions', docId));
```

1. addDoc

OK 2. deleteDoc

3. getDocs

4. serverTimestamp

正解: 2. deleteDoc

ヒント

削除対象はDocumentReferenceで指定します。

解説

deleteDocは指定したドキュメントを削除します。履歴を残したい業務データでは、物理削除ではなくstatus変更も検討します。

239 FSQ-159

削除

中級

全範囲

learningLogsコレクションの特定ドキュメントを削除したい。使うAPIは？

```
await deleteDoc(doc(db, 'learningLogs', docId));
```

1. addDoc

2. getDocs

OK 3. deleteDoc

4. serverTimestamp

正解: 3. deleteDoc

ヒント

削除対象はDocumentReferenceで指定します。

解説

deleteDocは指定したドキュメントを削除します。履歴を残したい業務データでは、物理削除ではなくstatus変更も検討します。

240 FSQ-160

削除

応用

全範囲

settingsコレクションの特定ドキュメントを削除したい。使うAPIは？

```
await deleteDoc(doc(db, 'settings', docId));
```

1. addDoc

2. getDocs

3. serverTimestamp

OK 4. deleteDoc

正解: 4. deleteDoc

ヒント

削除対象はDocumentReferenceで指定します。

解説

deleteDocは指定したドキュメントを削除します。履歴を残したい業務データでは、物理削除ではなくstatus変更も検討します。